PREFIX SUM

2026 SPRING

JINHO LEE

ECE, SNU

Accelerated
Intelligent
Systems Lab.

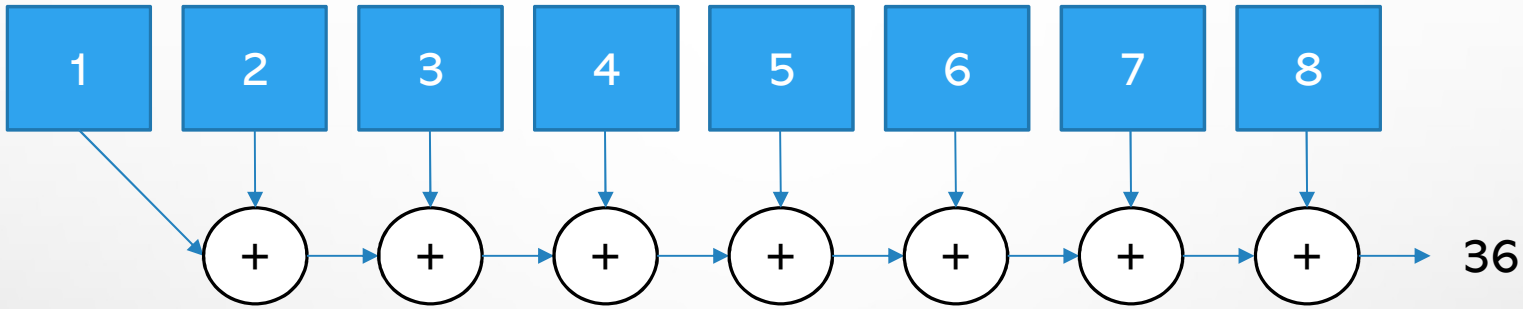
FLASHBACK – THE REDUCE PROBLEM

- We have already talked about parallelizing the reduce problem

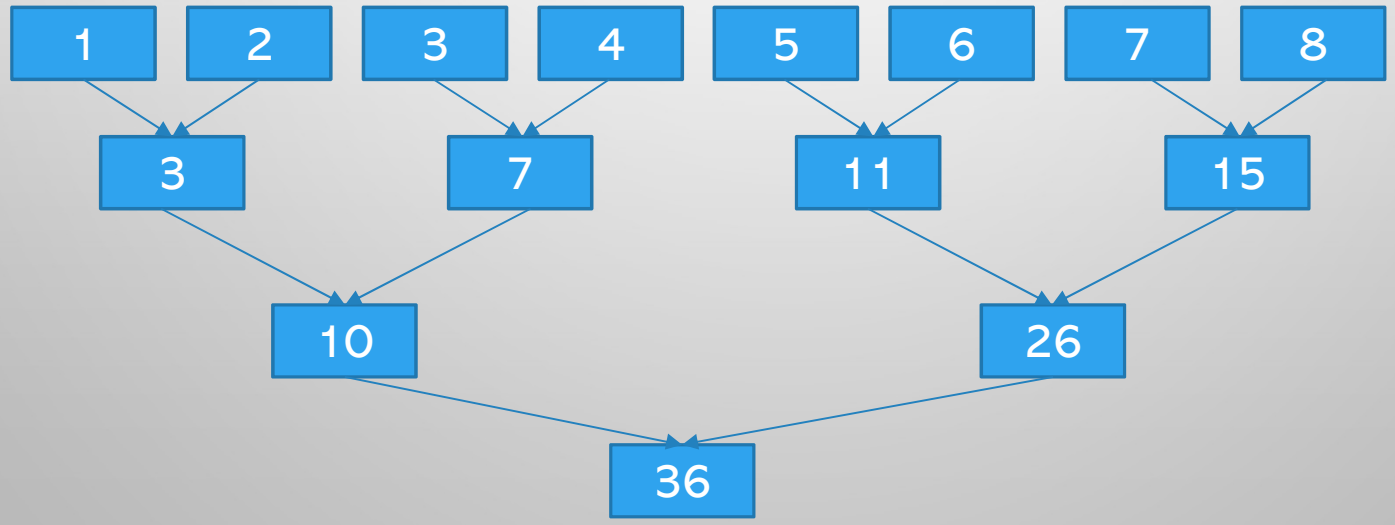
```
#include <iostream>
#include <thread>
#include <vector>

void worker(int* input, int start, int size,
int* output)
{
  for(int i=0;i<size;i++) {
    *(output) += input[i];
  }
}
```

FLASHBACK – THE REDUCE PROBLEM

- We have already talked about parallelizing the reduce problem



- It's about rethinking the algorithm from serial loop to a tree structure

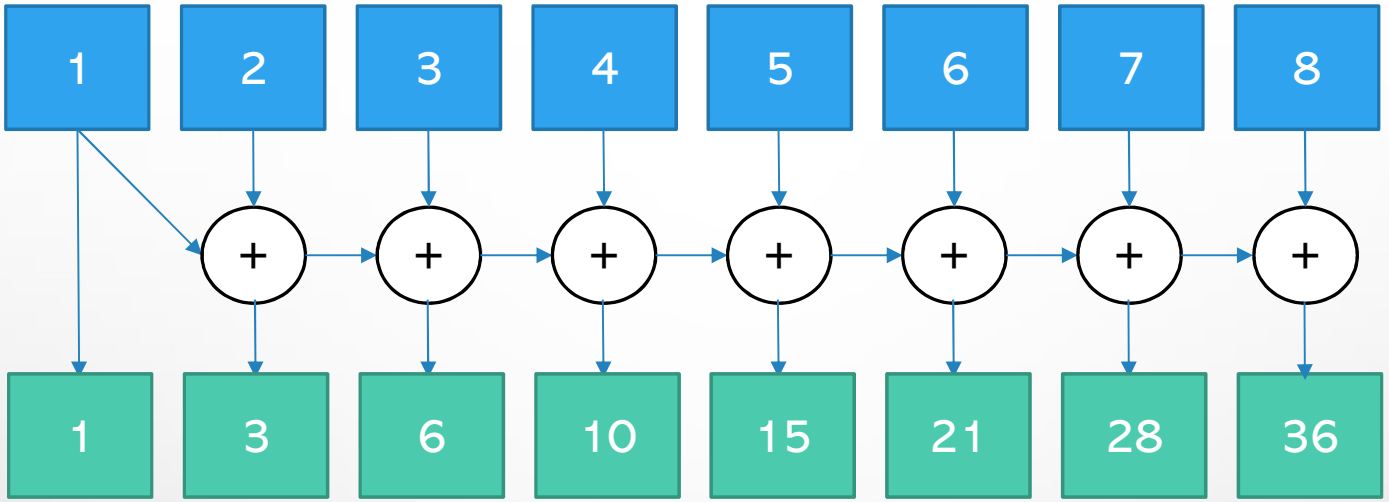


CONTENT

- We will not talk about GPUs, but about parallelization method

- Prefix Sum problem
- Kogge-Stone algorithm
- Blelloch algorithm

PREFIX SUM (SCAN)



- You have a 100cm bread
- Each person wants some portion (1cm, 2cm, 3cm, …)
- Where (from the beginning) do you have to cut?

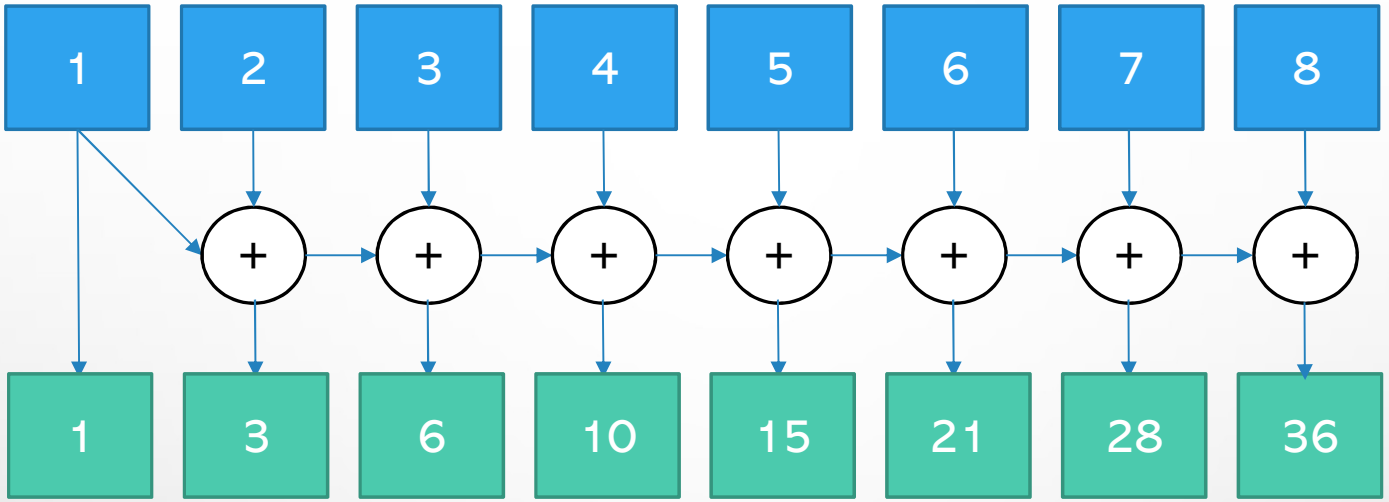
- This is the prefix sum, or prefix scan problem

PARALLEL PREFIX SUM (OR SCAN)

- Is used for many larger problems

- Quicksort (partition)
- Histogram
- Evaluate Polynomials
- Radix sort
- Search for regular expressions
- Many more…

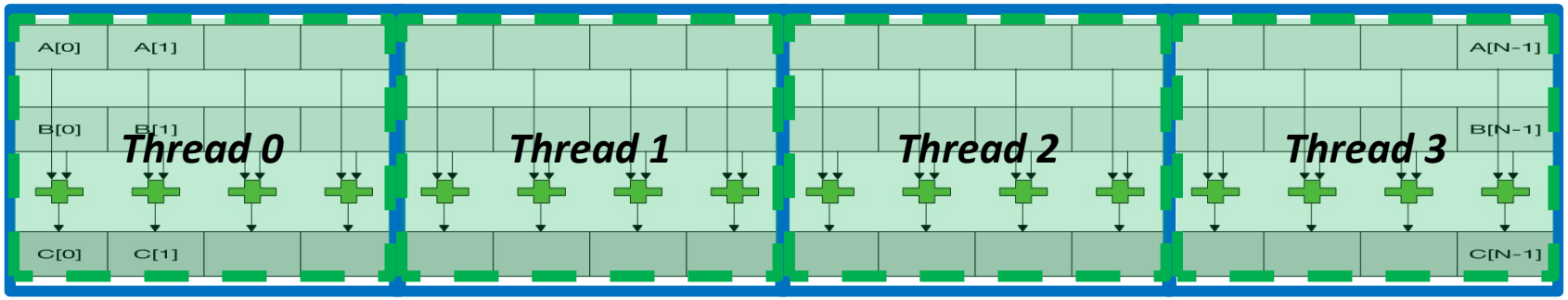
PREFIX SUM (SCAN)



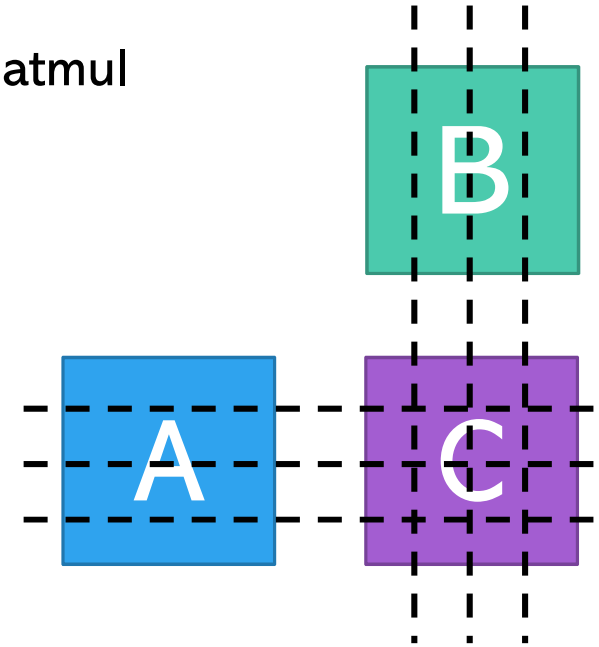
- Number of additions is 7 – O(N) complexity
  - Equal to the reduce problem
- Can we do this in parallel?

NAÏVE WAY…

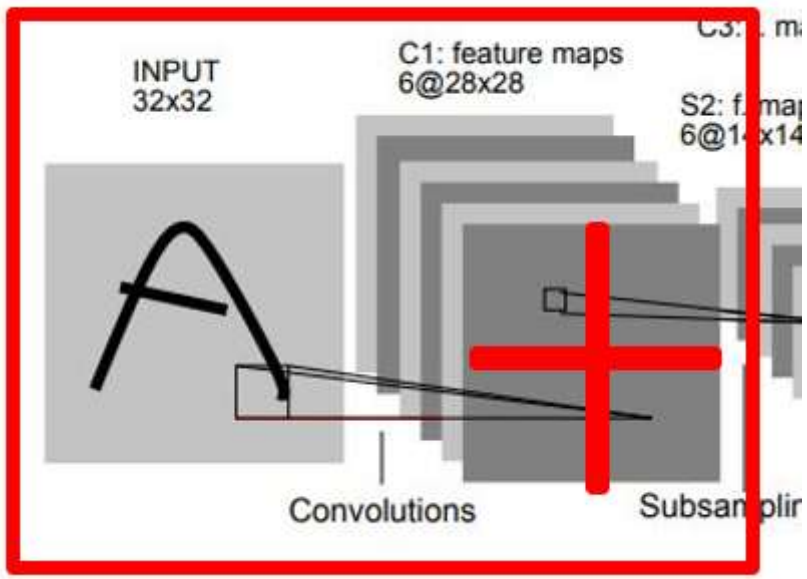
- In the naïve ways we have seen so far…
- Vector add



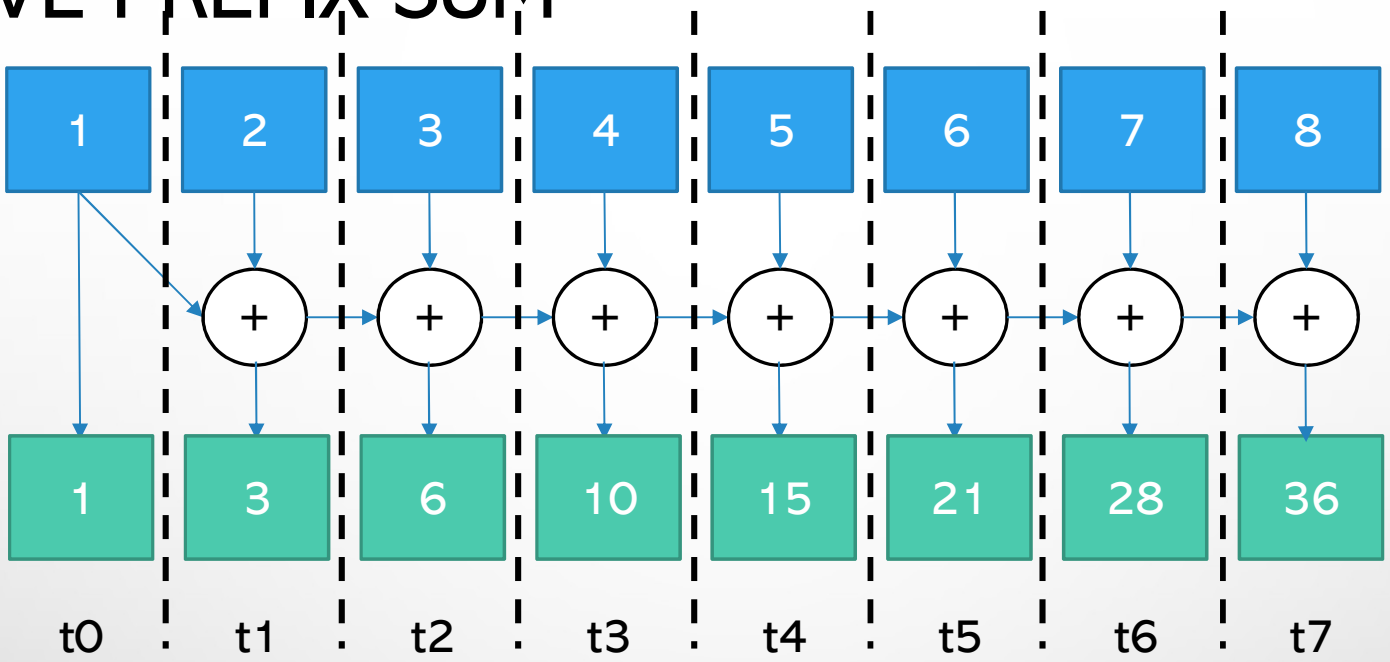
- Matmul



- DNN

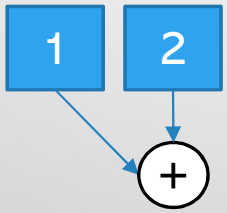


NAÏVE PREFIX SUM

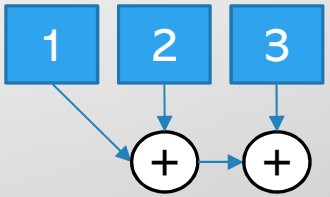


Thread 0: $\sum_{i=0}^{0} In[i]$

Thread 1: $\sum_{i=0}^{1} In[i]$



Thread 2: $\sum_{i=0}^{2} In[i]$



Thread k: $\sum_{i=0}^{k} In[i]$

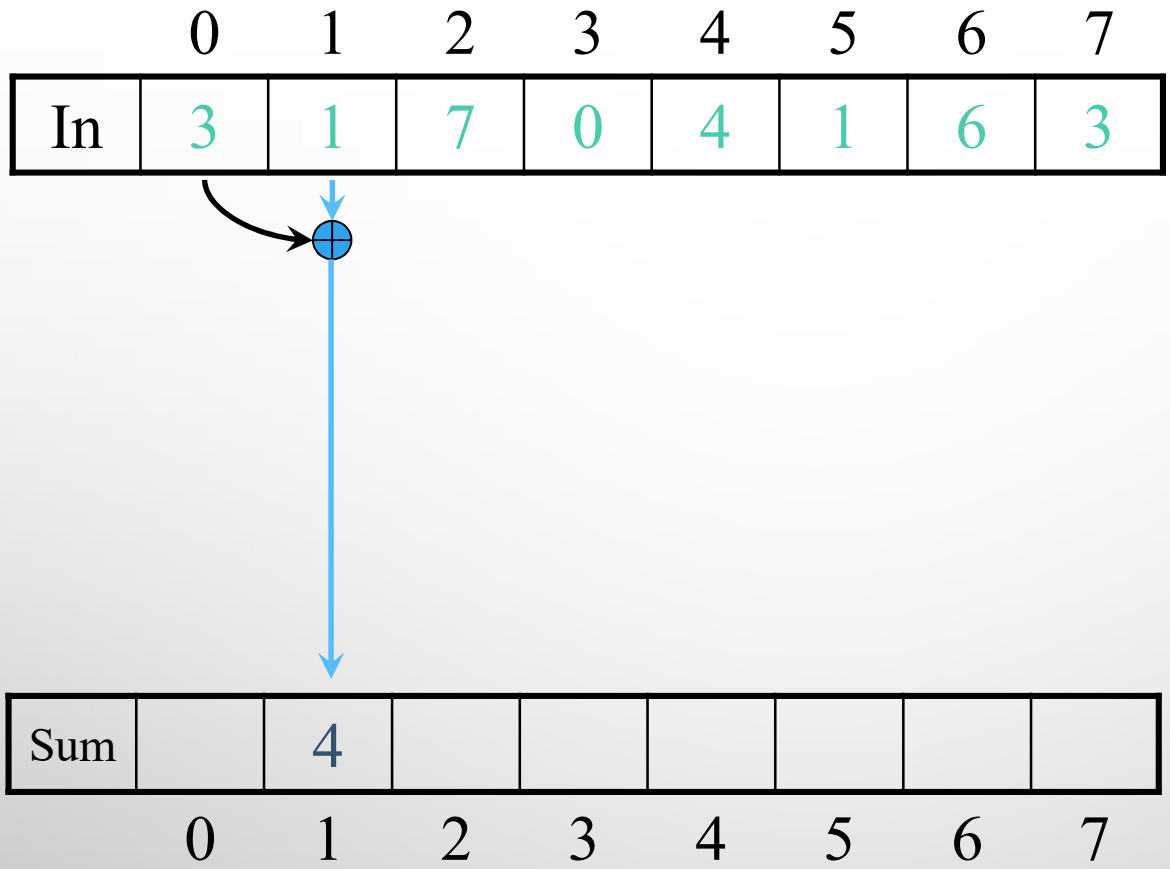
- Parallelization is so easy,
  as long as you don't care about the speed..
- But we will start from here to build a better one

NAÏVE ALGORITHM – THREAD 0

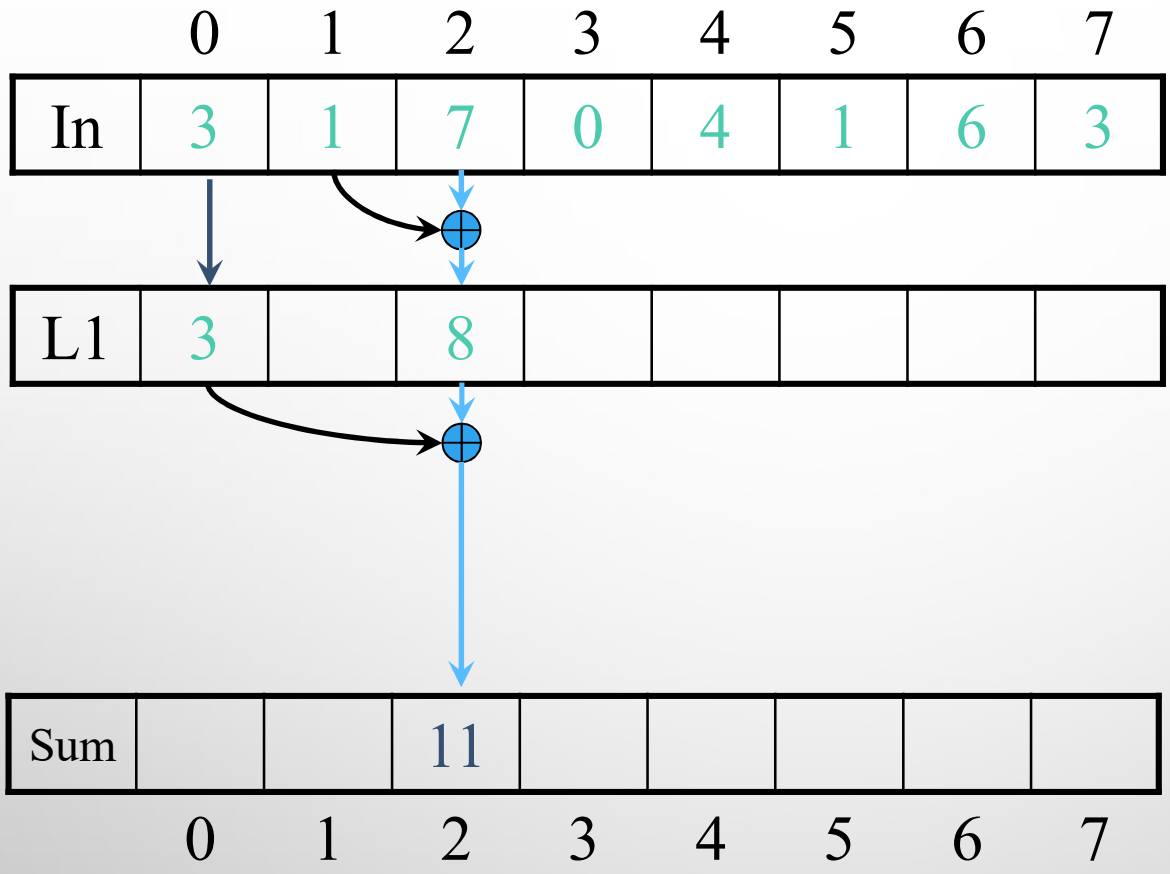
	0	1	2	3	4	5	6	7
In	3	1	7	0	4	1	6	3

	0	1	2	3	4	5	6	7
Sum	3							

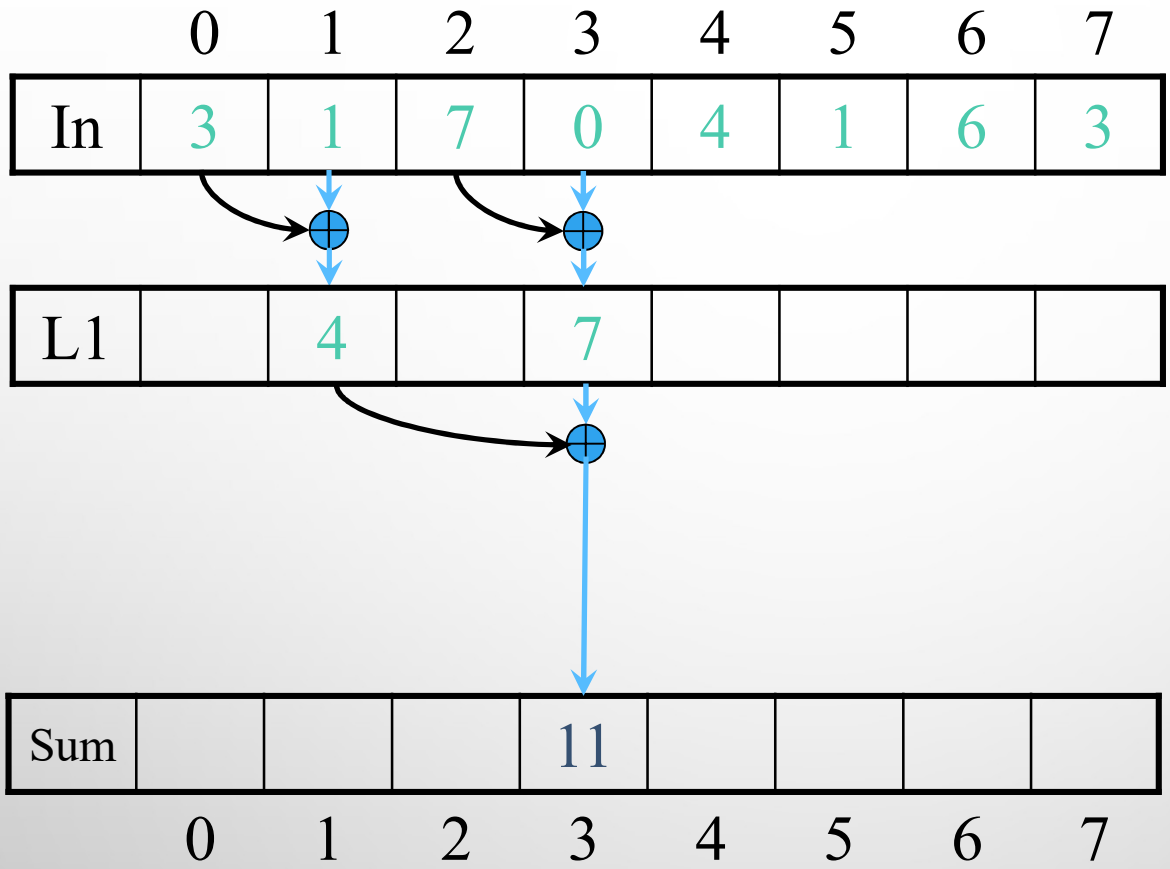
NAÏVE ALGORITHM – THREAD 1



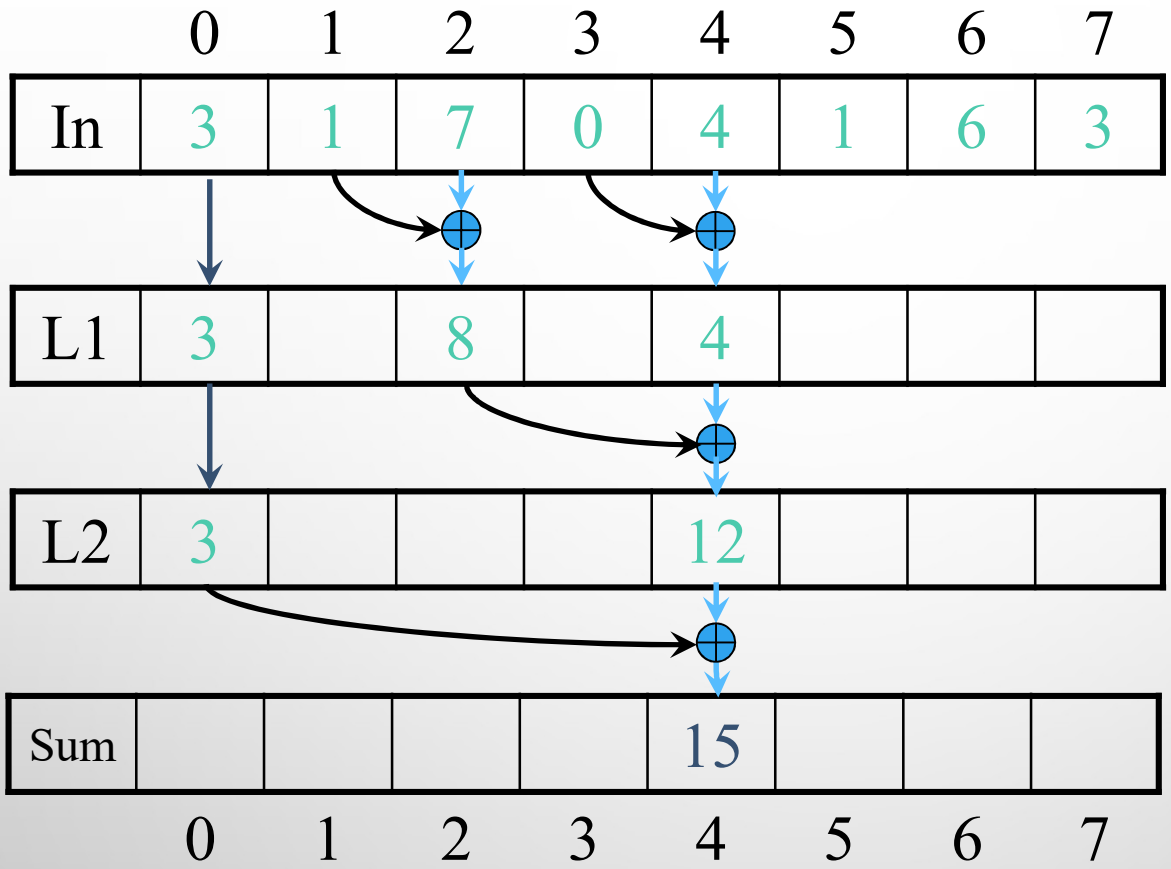
NAÏVE ALGORITHM – THREAD 2



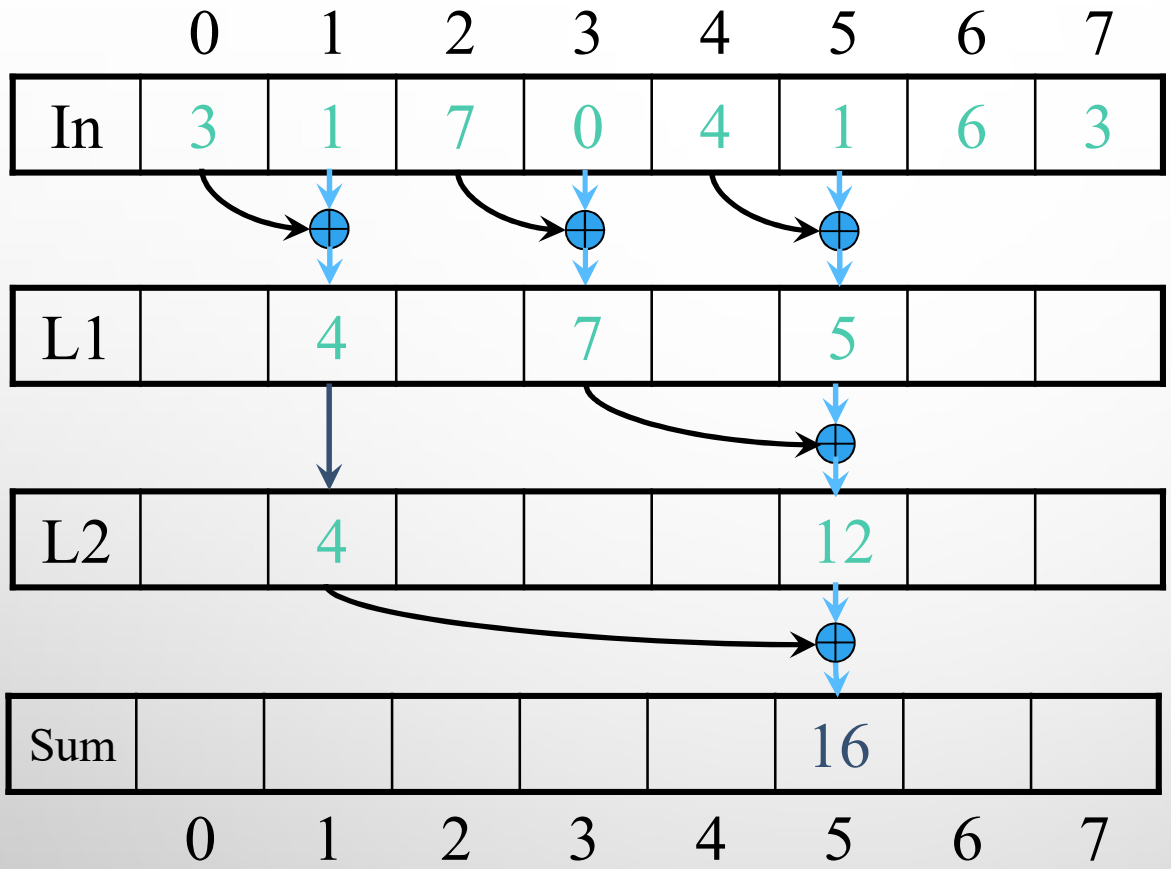
NAÏVE ALGORITHM – THREAD 3



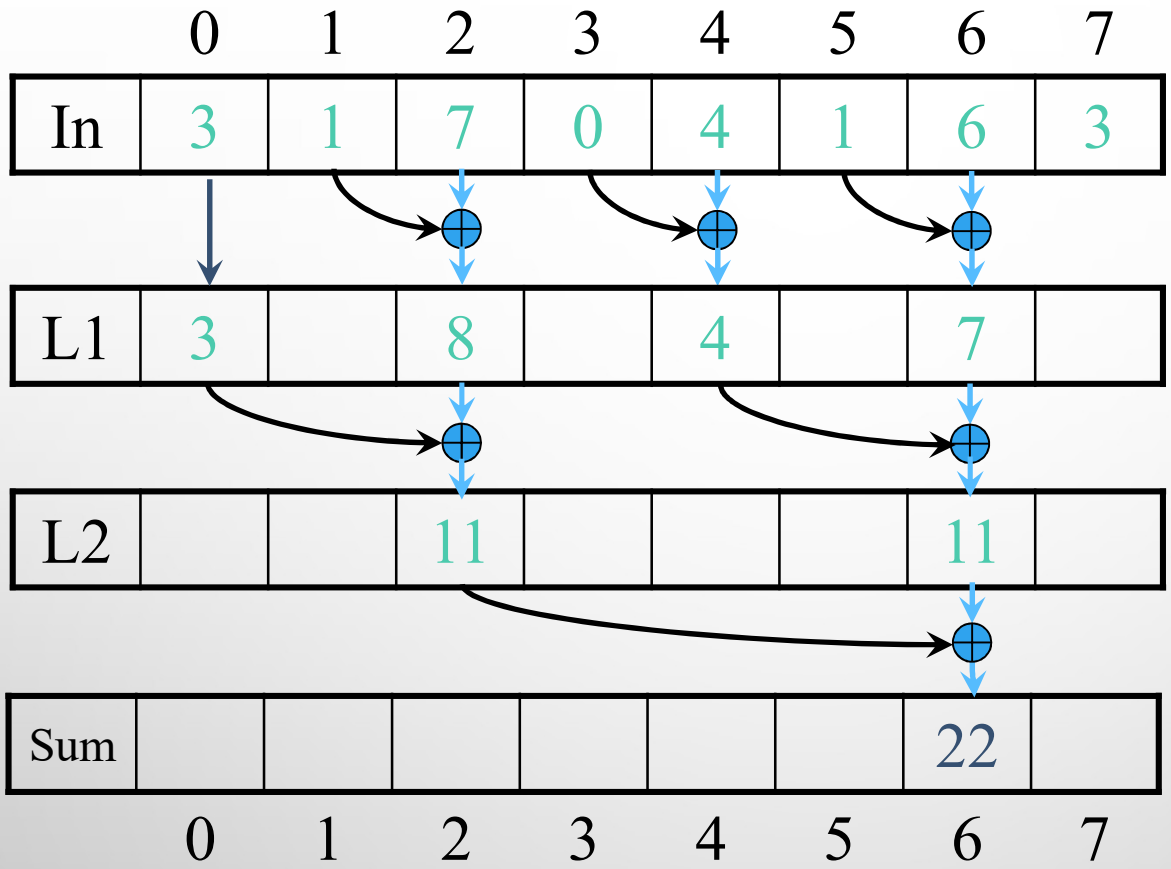
NAÏVE ALGORITHM – THREAD 4



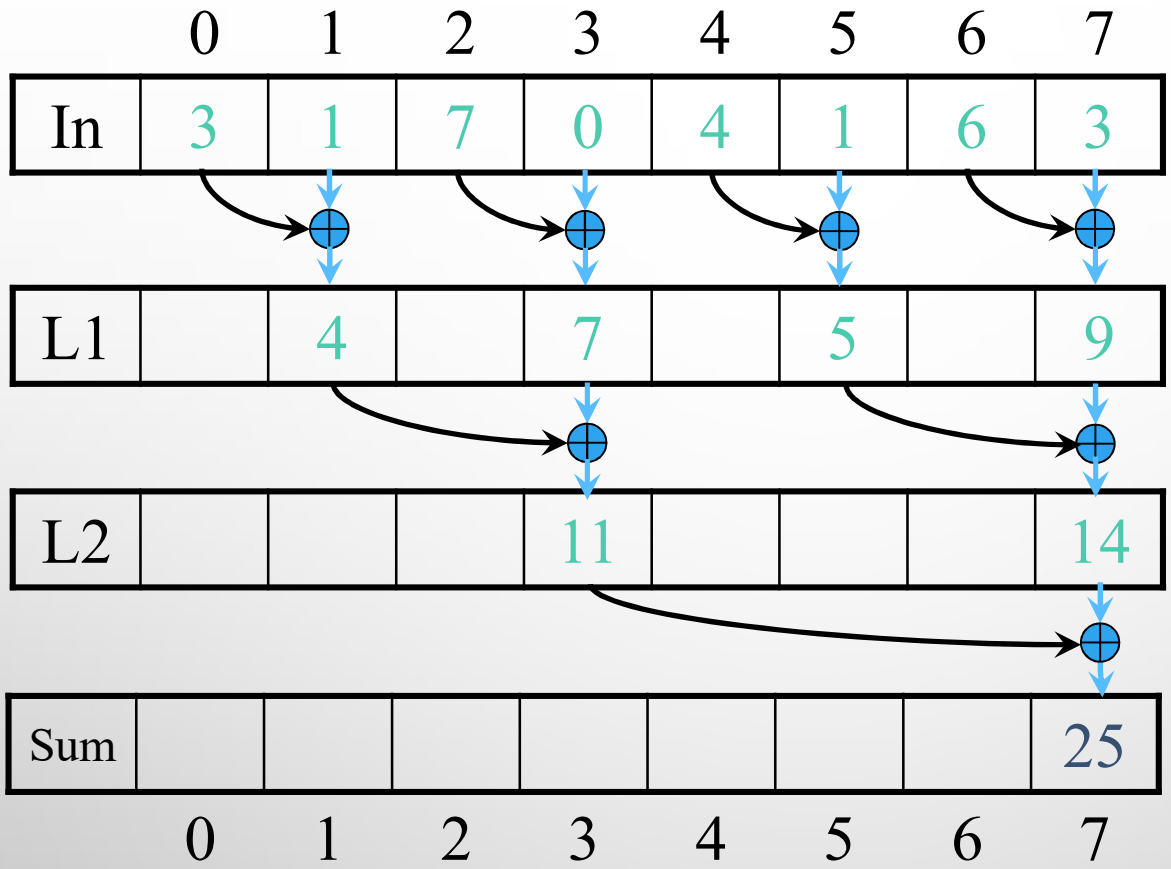
NAÏVE ALGORITHM – THREAD 5



NAÏVE ALGORITHM – THREAD 6

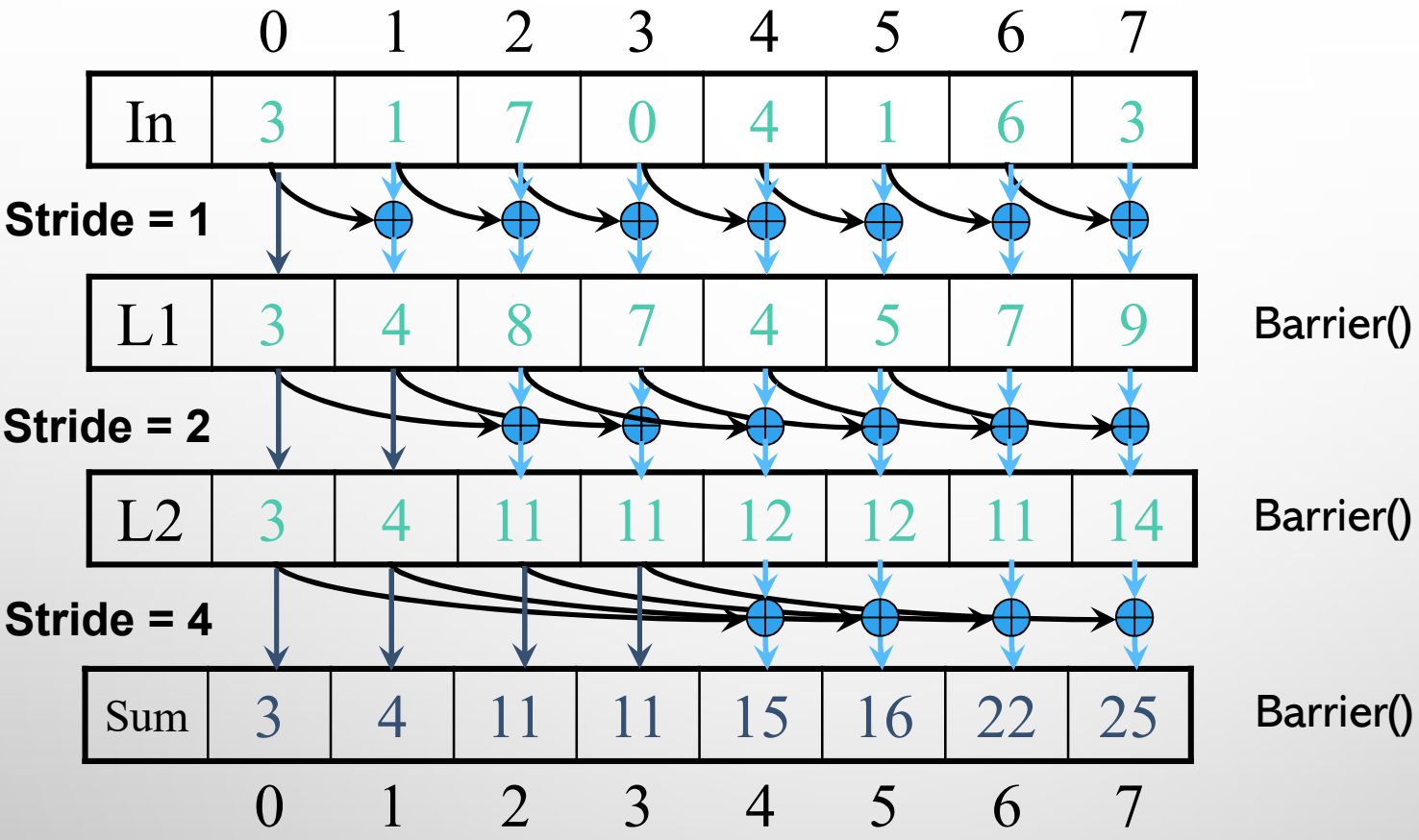


NAÏVE ALGORITHM – THREAD 7



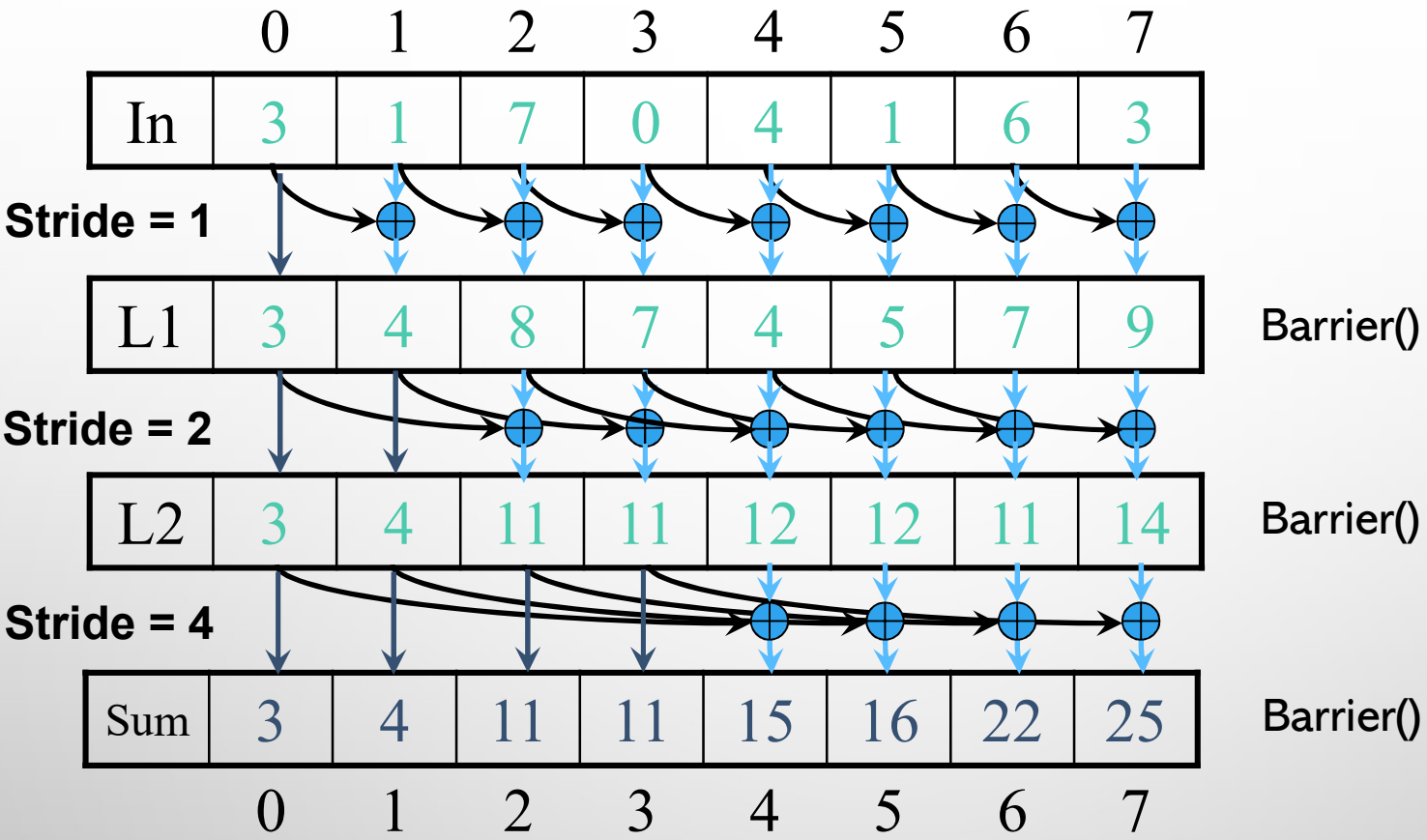
- See the pattern?
- → Same computations appear multiple times

KOGGE-STONE ALGORITHM



- Calculate one level, and place a barrier
- In k^{th} iteration, $A_{k+1}[i] = A_k[i-2^k] + A_k[i]$
- With infinitely many cores, this is done in O(logN) steps

KOGGE-STONE ALGORITHM



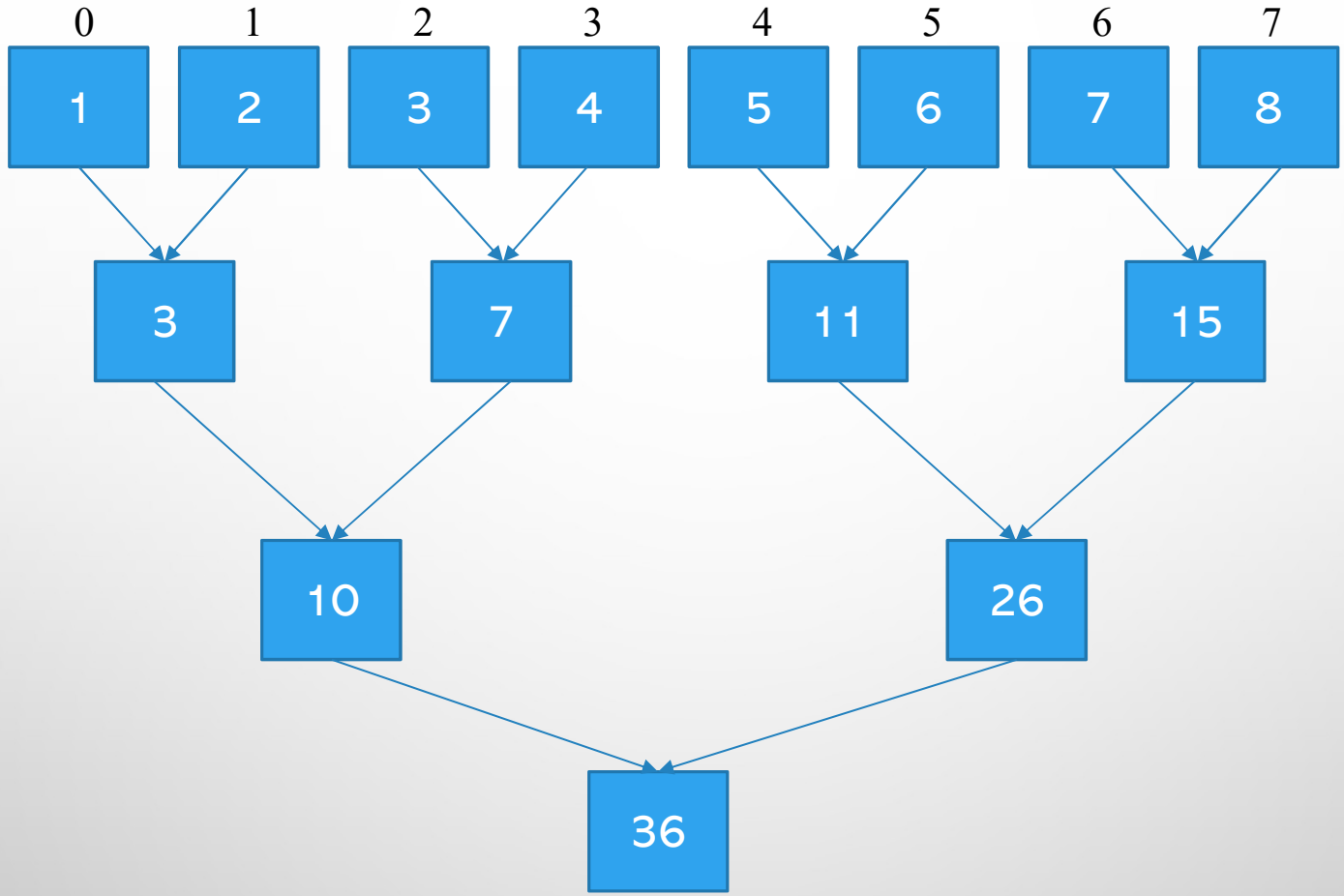
- With finite *P* cores
  - N-1 + N-2 + N-4 + … + N-N/2 = O(NlogN) additions
  - Speedup = $O(\frac{NlogN}{P})$
- Memory requirement: ~~NlogN~~ → 2N (with double buffering technique)

BLELLOCH ALGORITHM

- Kogge-Stone algorithm achieves good performance with massive amount of hardware
  - e.g., GPUs/supercomputers
- At the cost of parallelization, it adds logN time more work

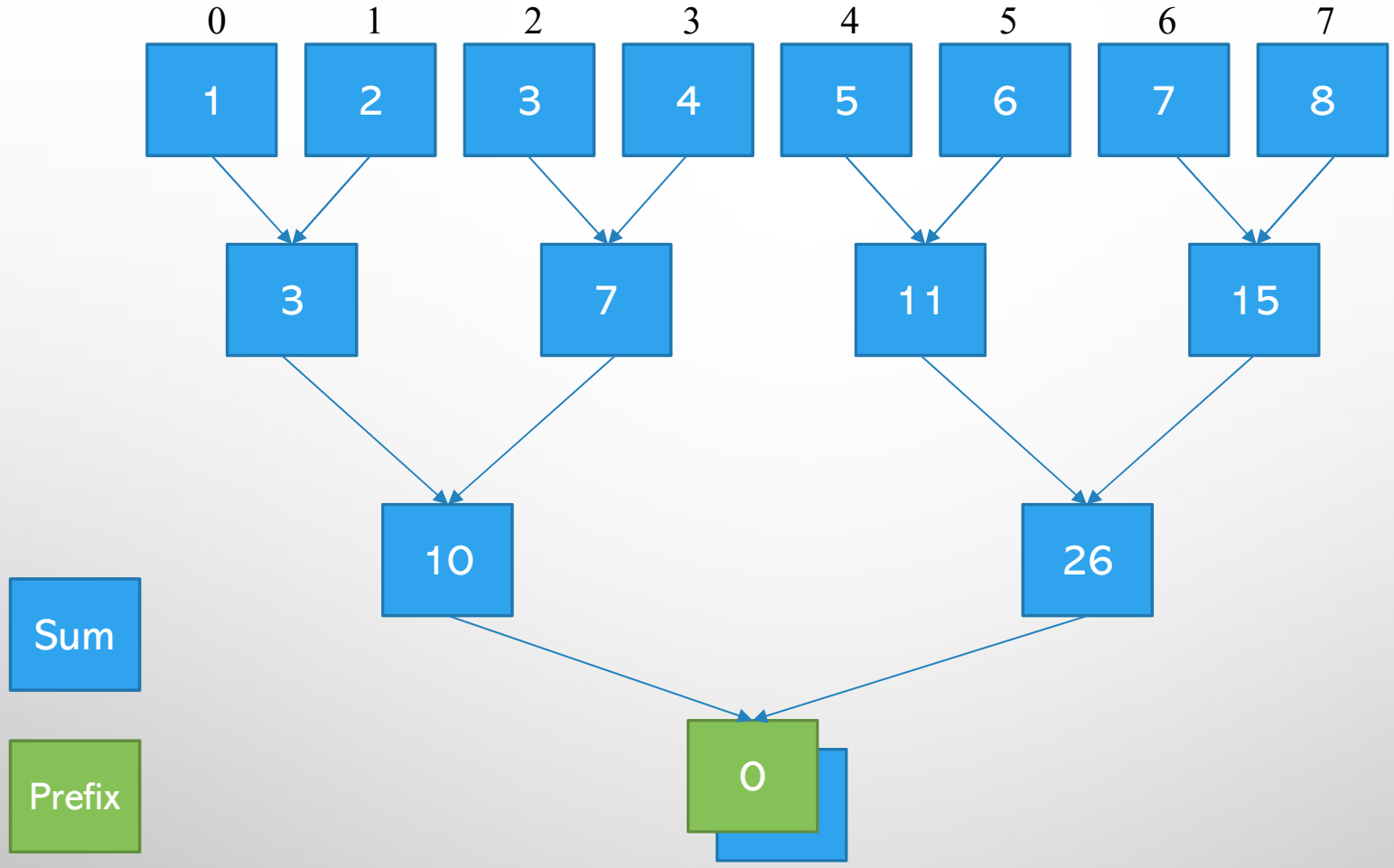
- Blelloch algorithm performs parallel reduction at the same complexity O(N)
- Use a balanced binary tree
  - Not the real tree data structure, but a computation pattern

BLELLOCH PARALLEL SCAN



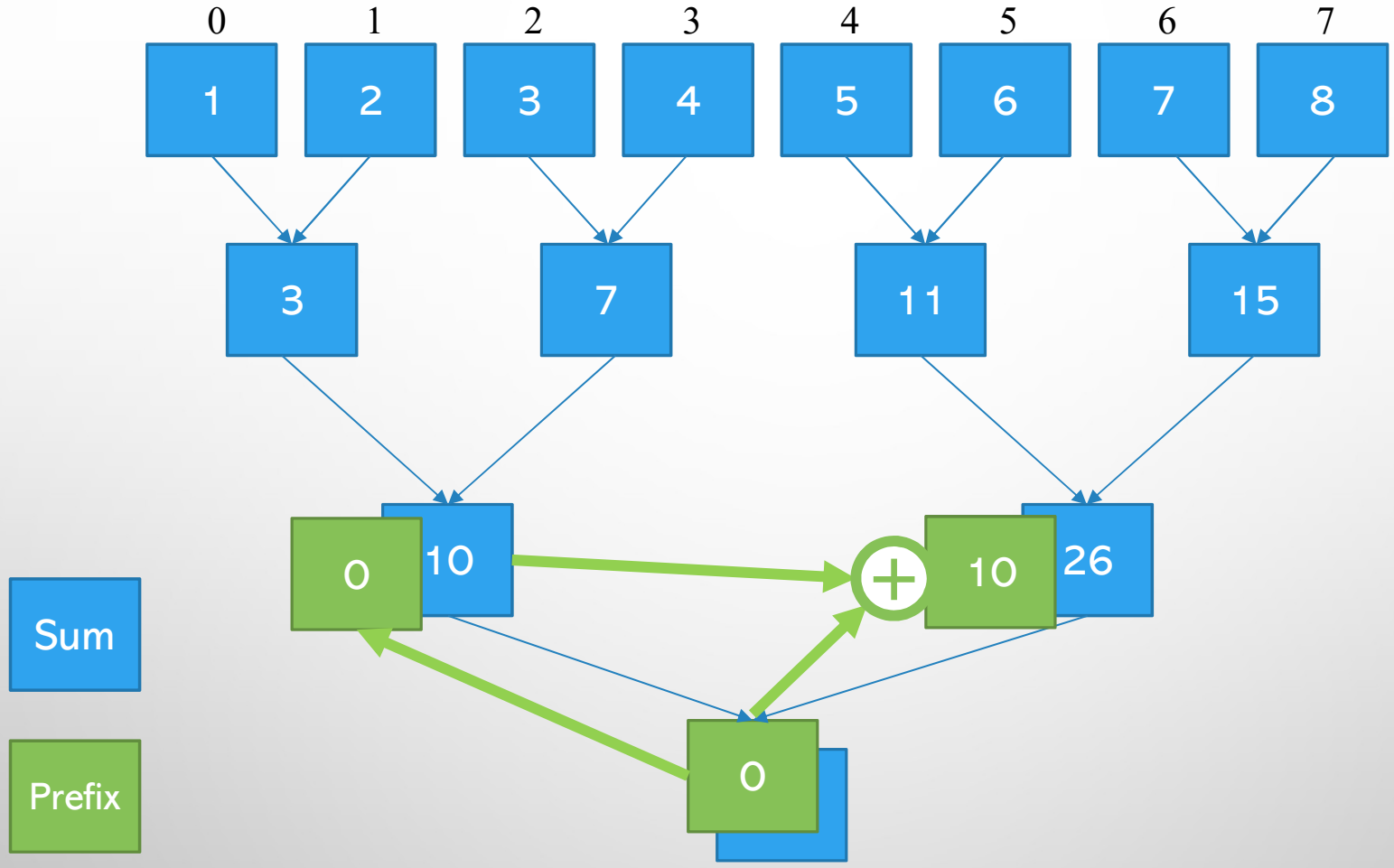
- Phase 1
  - Same as the parallel reduce

BLELLOCH PARALLEL SCAN



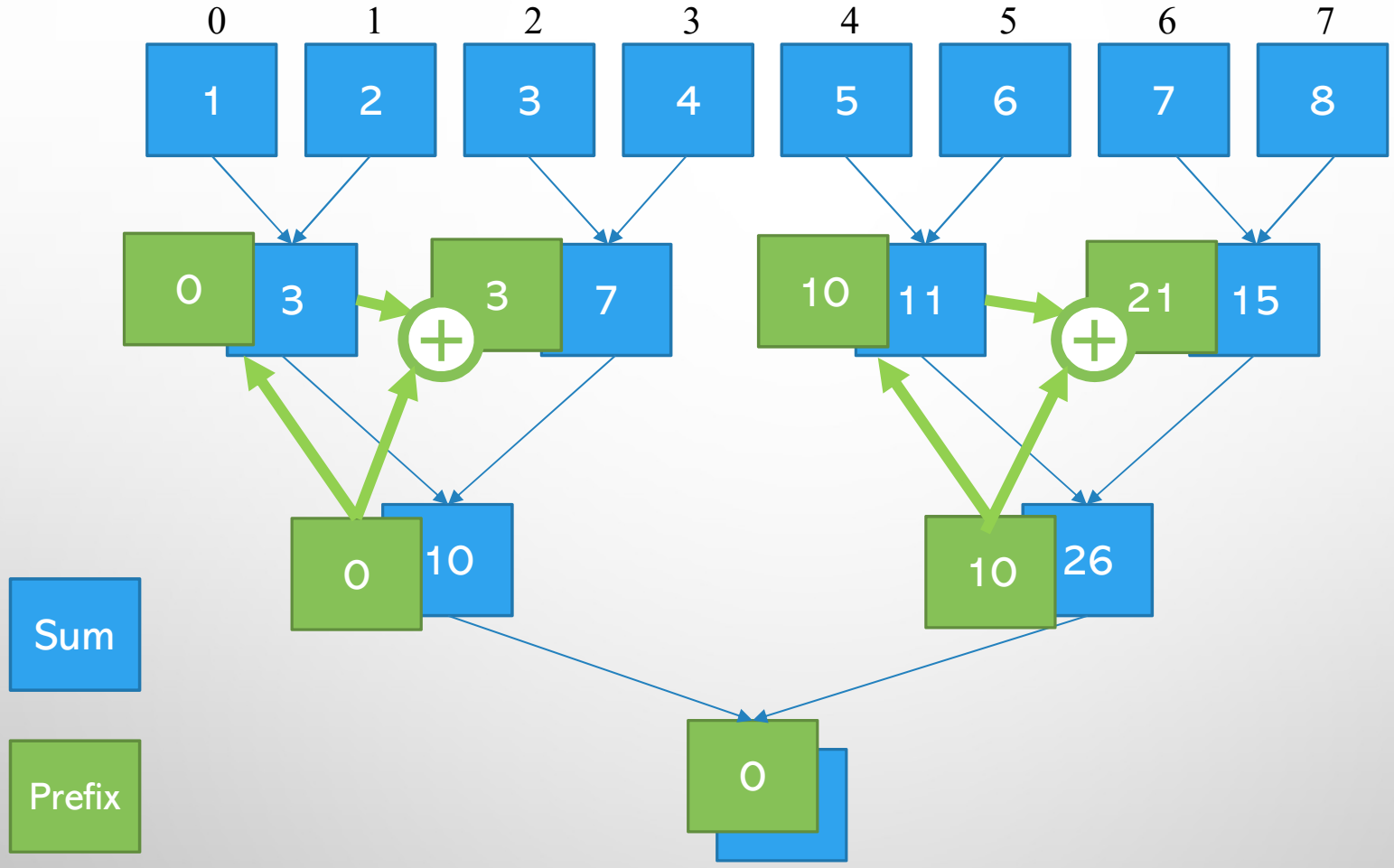
- Phase 2
  - Topdown computation
  - Start from 0 (identity for + op)

BLELLOCH PARALLEL SCAN



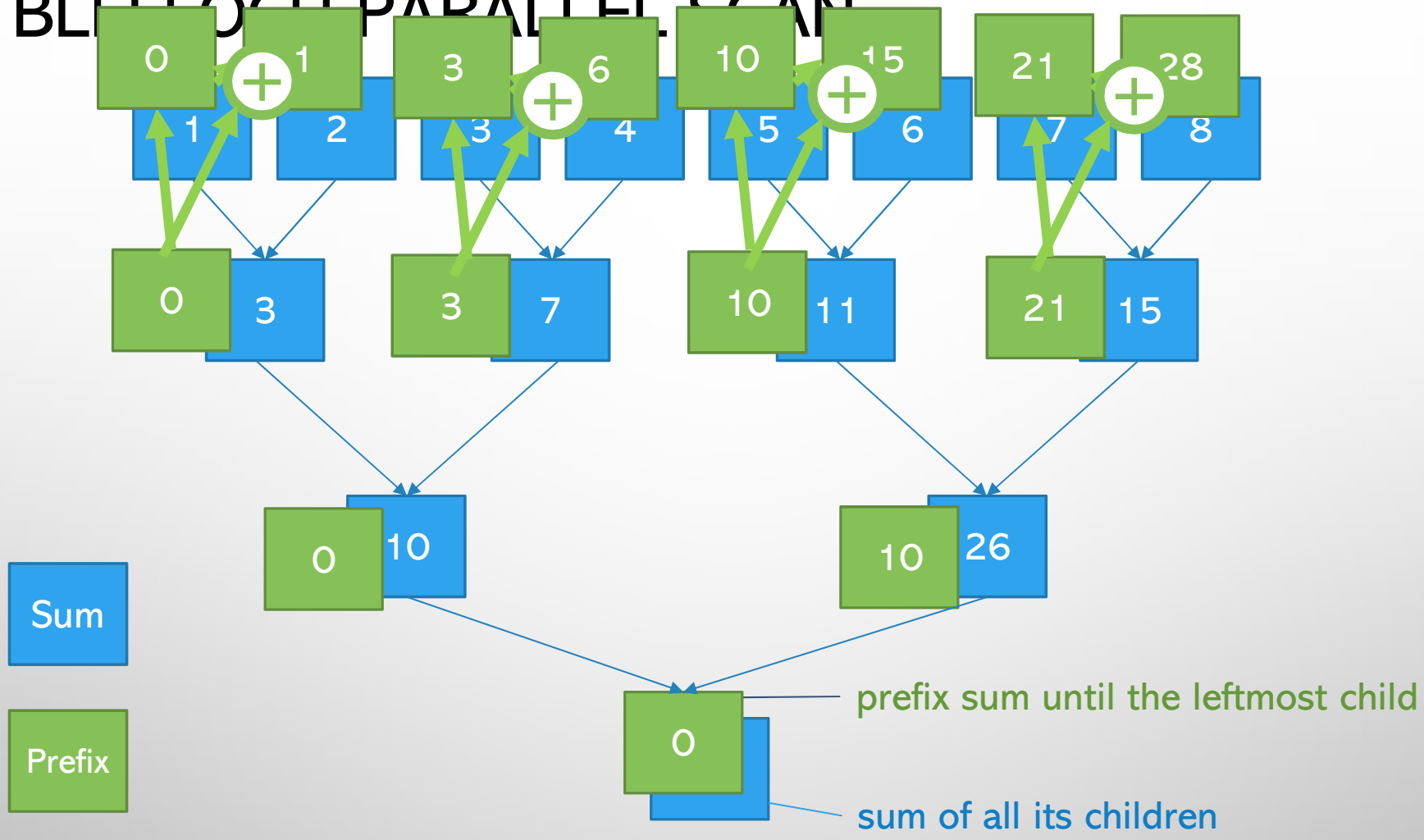
- Prefix[Left(v)] = Prefix[v]
- Prefix[Right(v)] = Prefix[v] + Sum[Left(v)]

BLELLOCH PARALLEL SCAN



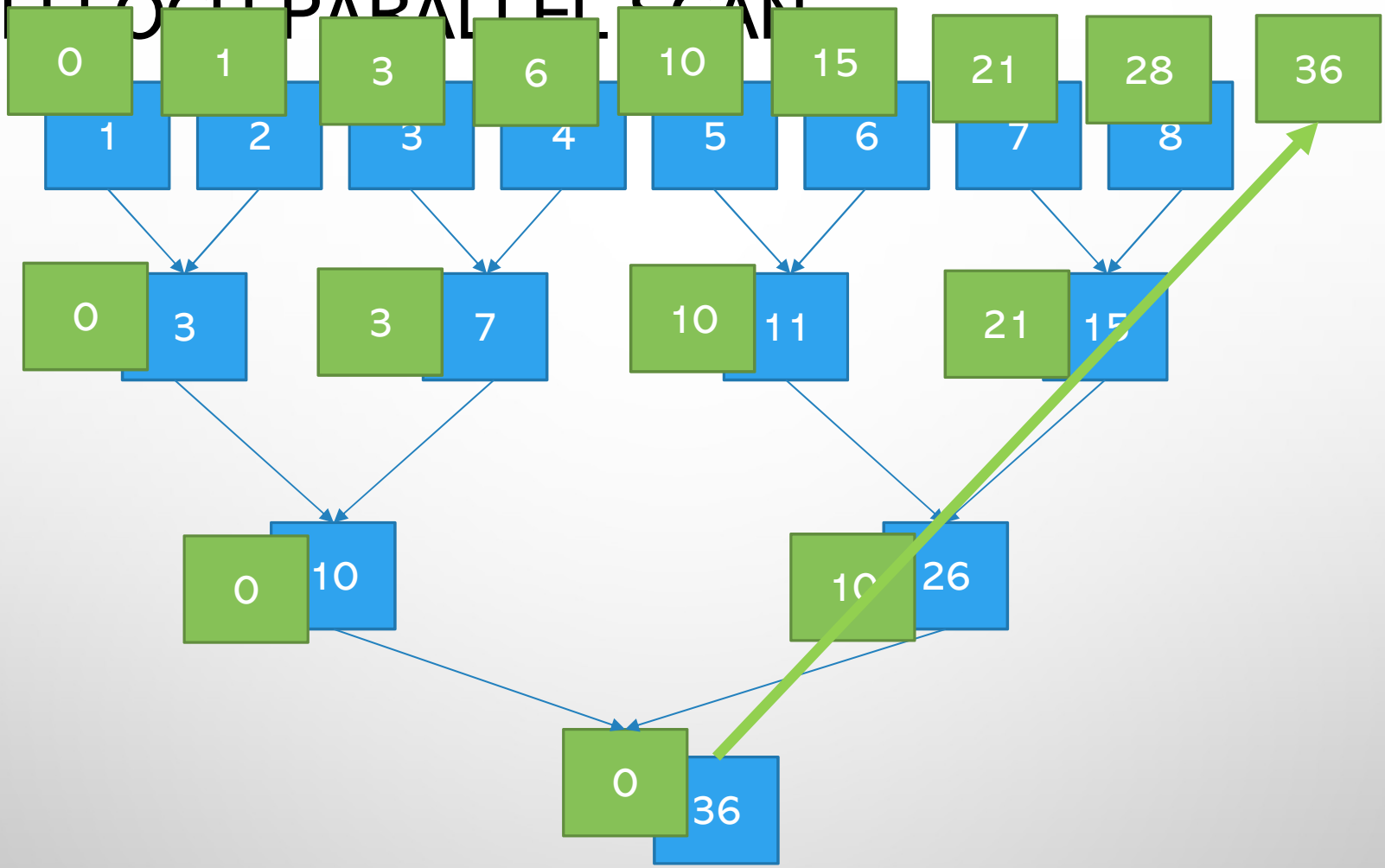
- Prefix[Left(v)] = Prefix[v]
- Prefix[Right(v)] = Prefix[v] + Sum[Left(v)]

BLELLOCH PARALLEL SCAN



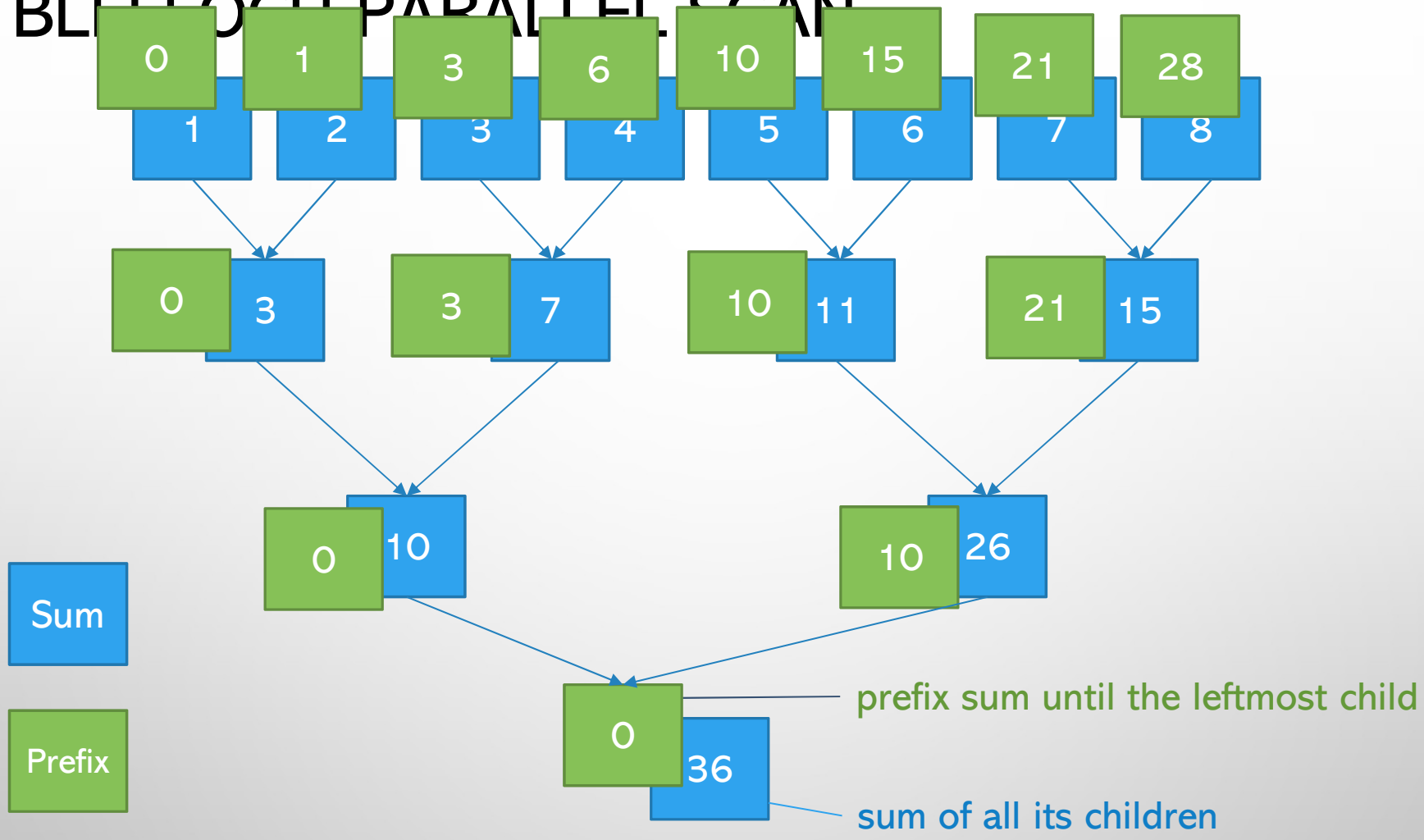
- Prefix[Left(v)] = Prefix[v]
- Prefix[Right(v)] = Prefix[v] + Sum[Left(v)]

BLELLOCH PARALLEL SCAN



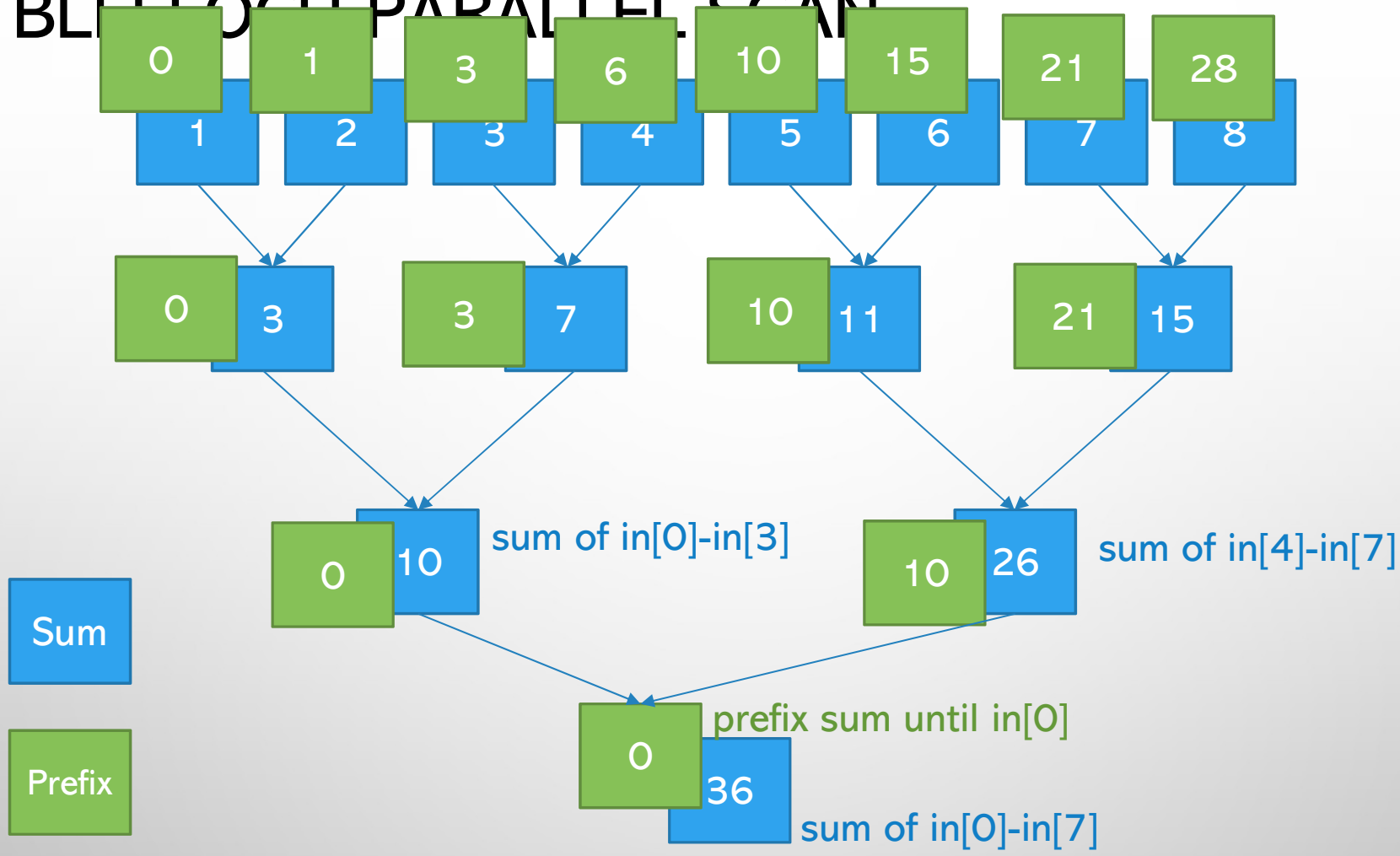
- Add the global sum

BLELLOCH PARALLEL SCAN



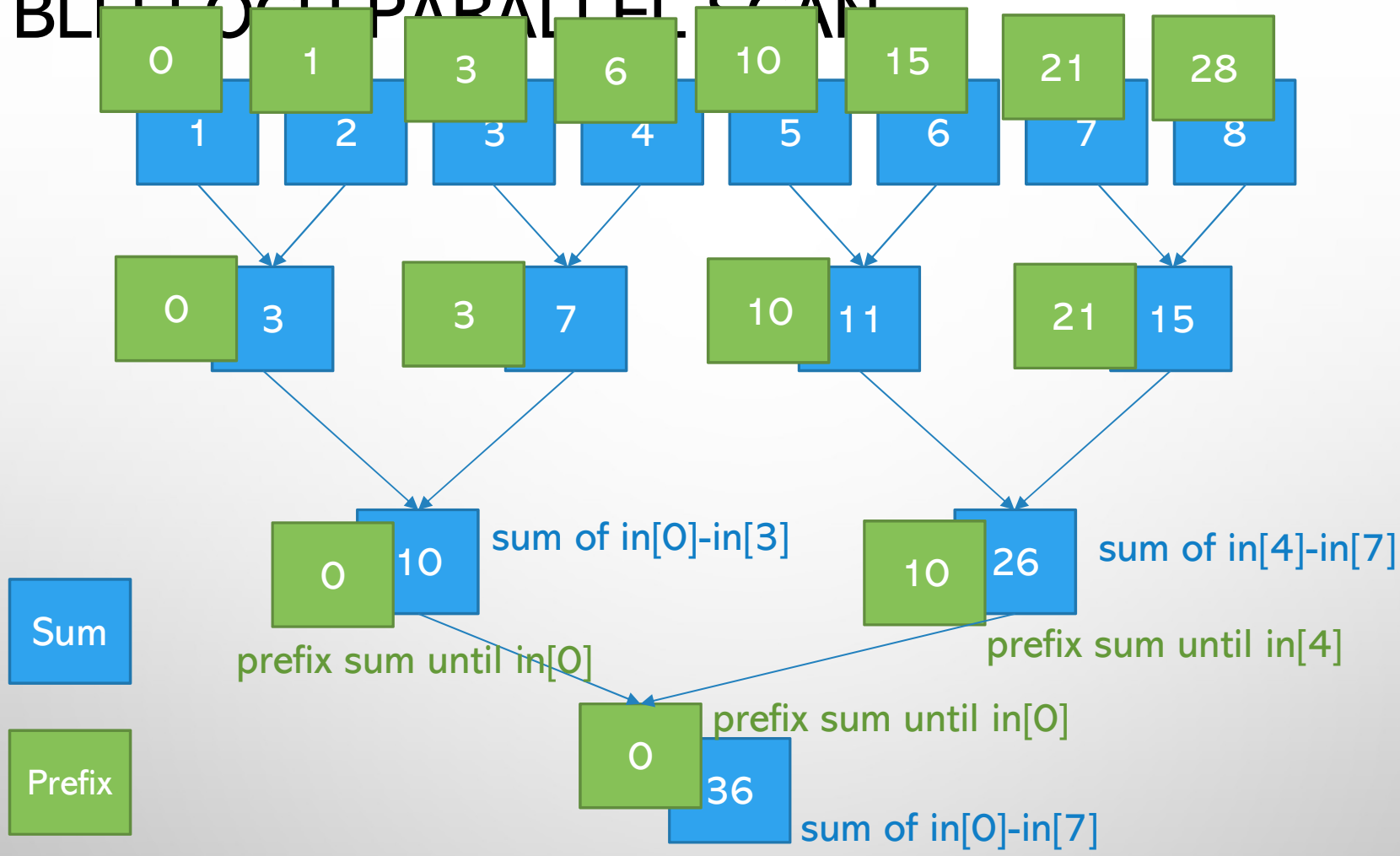
- Prefix[Left(v)] = Prefix[v]
- Prefix[Right(v)] = Prefix[v] + Sum[Left(v)]

BLELLOCH PARALLEL SCAN



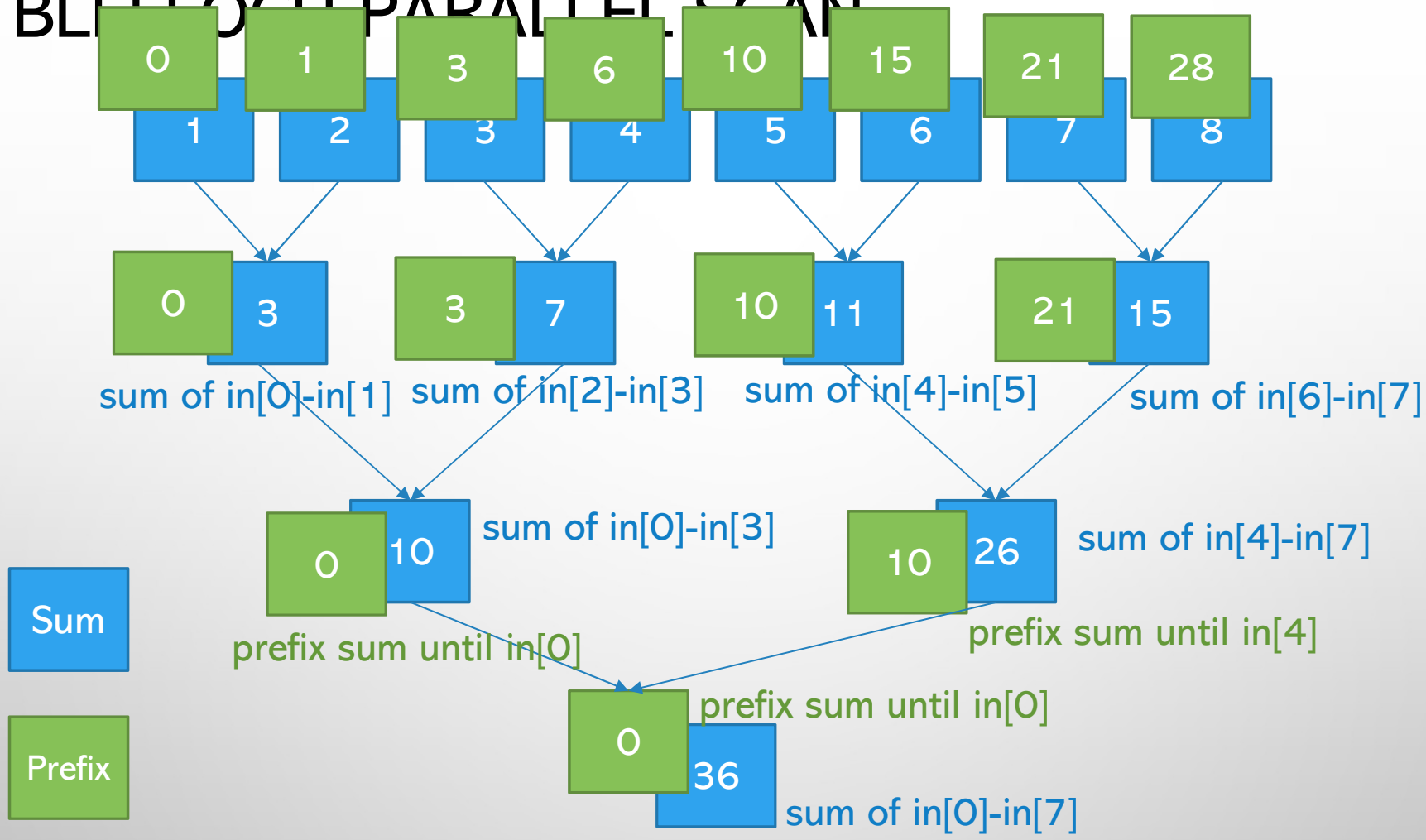
- Prefix[Left(v)] = Prefix[v]
- Prefix[Right(v)] = Prefix[v] + Sum[Left(v)]

BLELLOCH PARALLEL SCAN



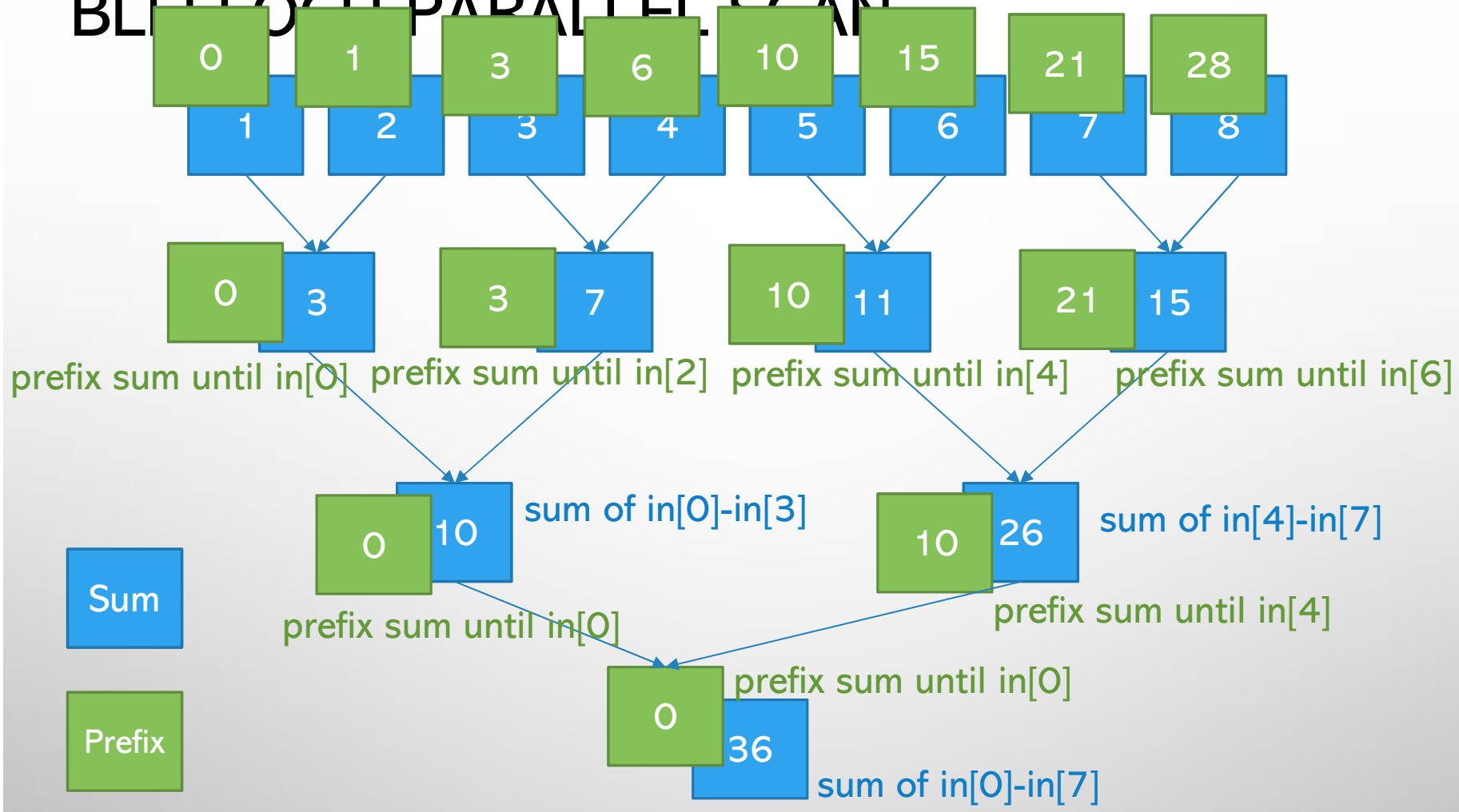
- Prefix[Left(v)] = Prefix[v]
- Prefix[Right(v)] = Prefix[v] + Sum[Left(v)]

BLELLOCH PARALLEL SCAN



- Prefix[Left(v)] = Prefix[v]
- Prefix[Right(v)] = Prefix[v] + Sum[Left(v)]

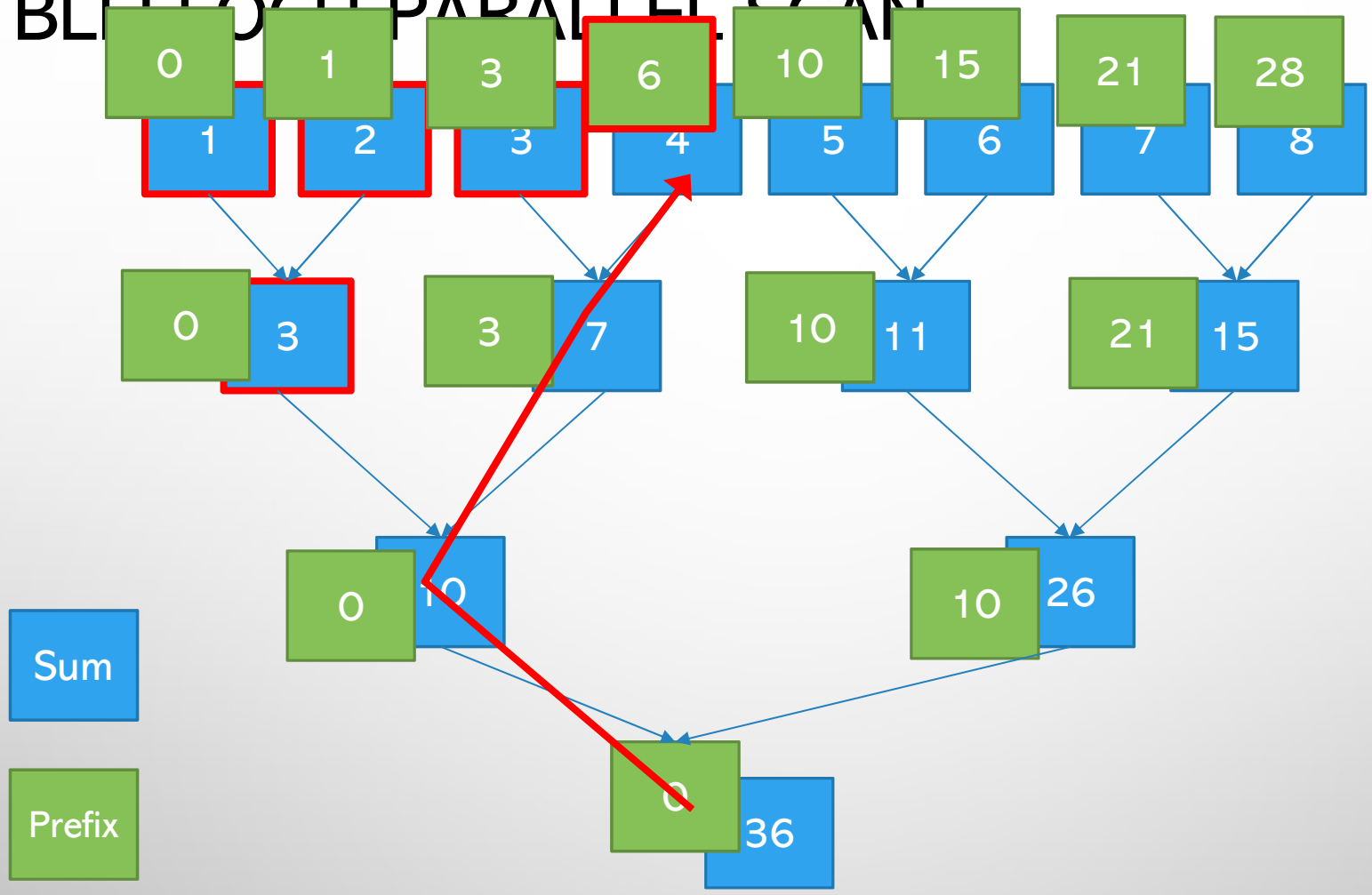
BLELLOCH PARALLEL SCAN



- Interpretation:

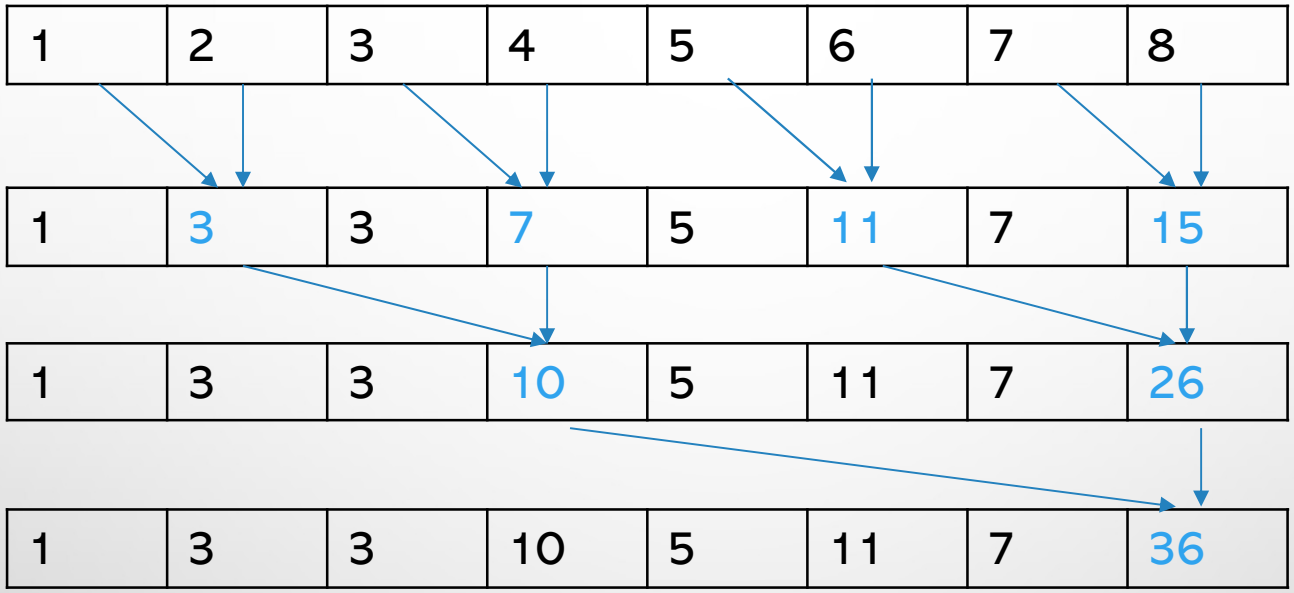
  When you take a right child, you're adding everything on the left subtree

BLELLOCH PARALLEL SCAN



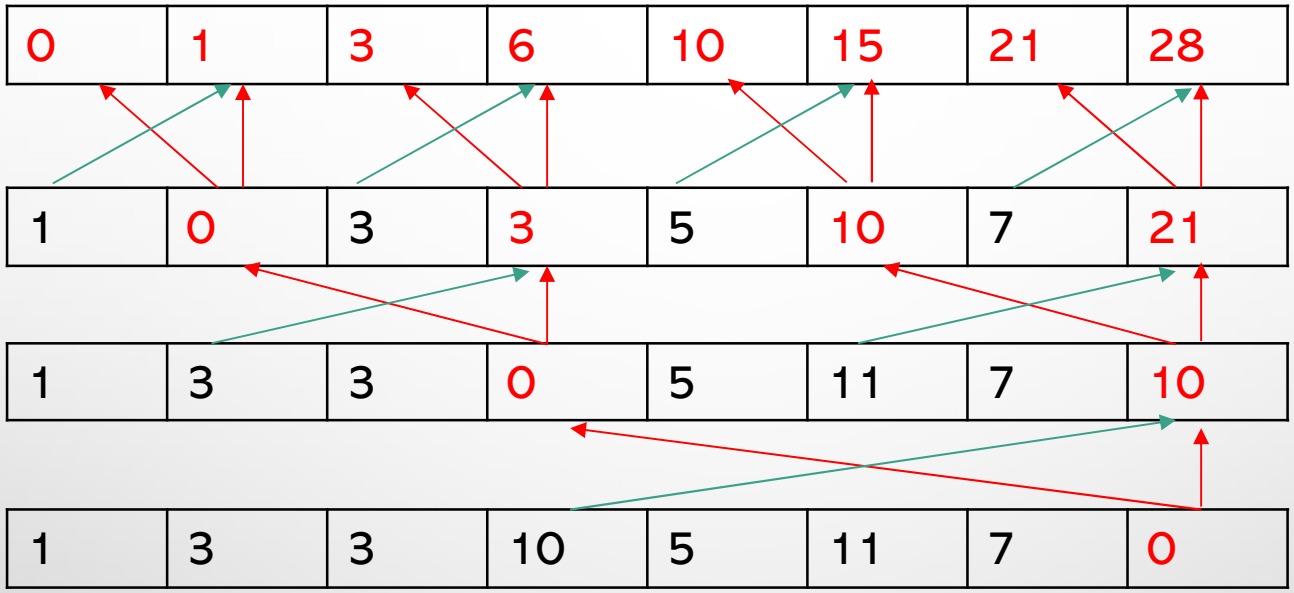
- Interpretation:
  When you take a right child, you're adding everything on the left subtree
- Prefix[3] = Sum[0,1] + Sum[2]

BLELLOCH ALGORITHM – ARRAY VIEW



- No need for extra memory
- First Phase – calculate Sum[] array
  - Observation: Every other element will not be needed later. Overwrite.

BLELLOCH ALGORITHM – ARRAY VIEW



Propagation of Prefix[v]

Propagation of Sum[Left(v)]

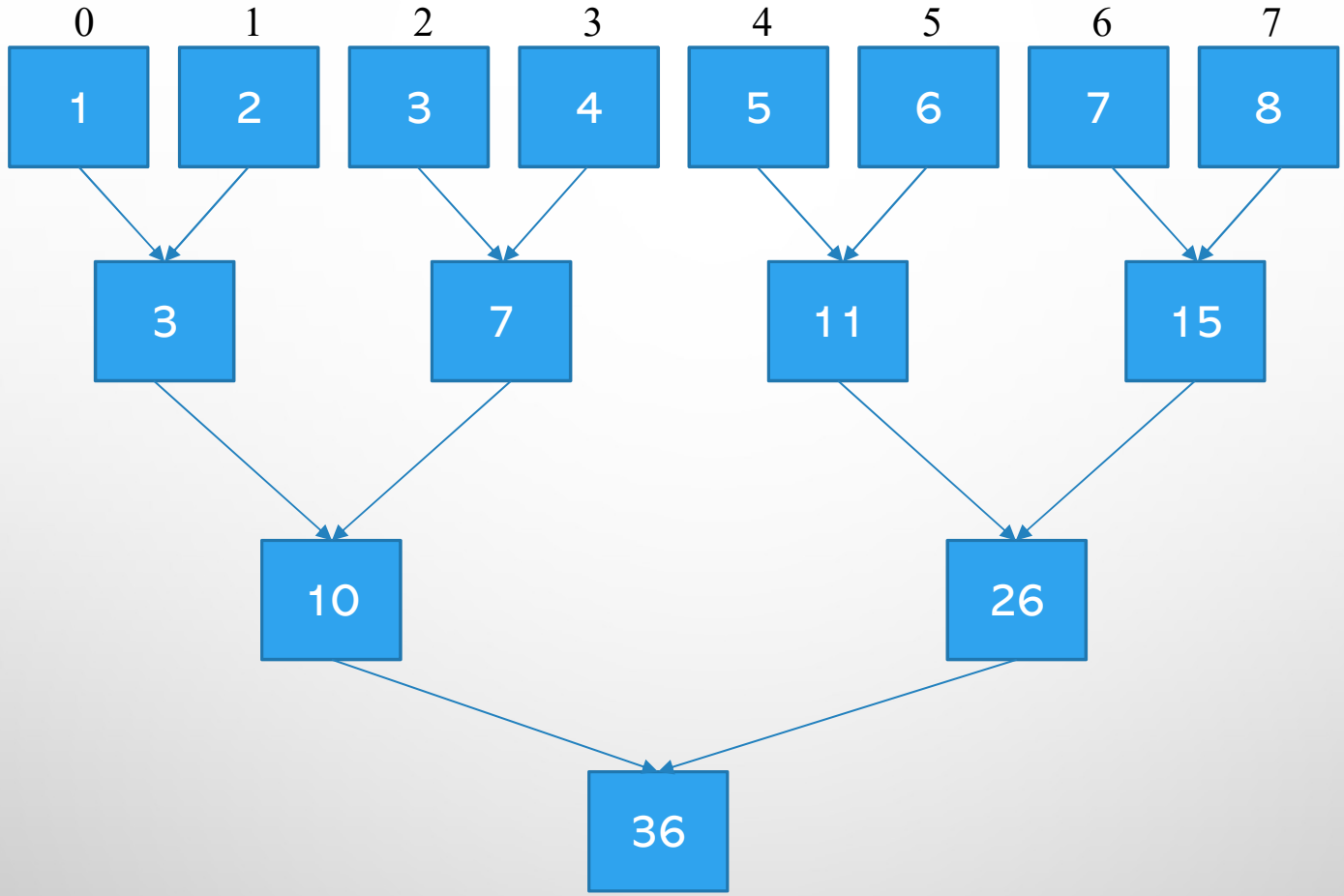
- No need for extra memory
- Second Phase: Calculate prefix[]
  - Similar observation: Sum[] will not be needed after being added

BLELLOCH ALGORITHM

- Time : 2logN steps – O(logN)
- Computation: 2(N-1) = O(N)
  - With P processors, O(N/P)
- Comparison with Kogge-Stone

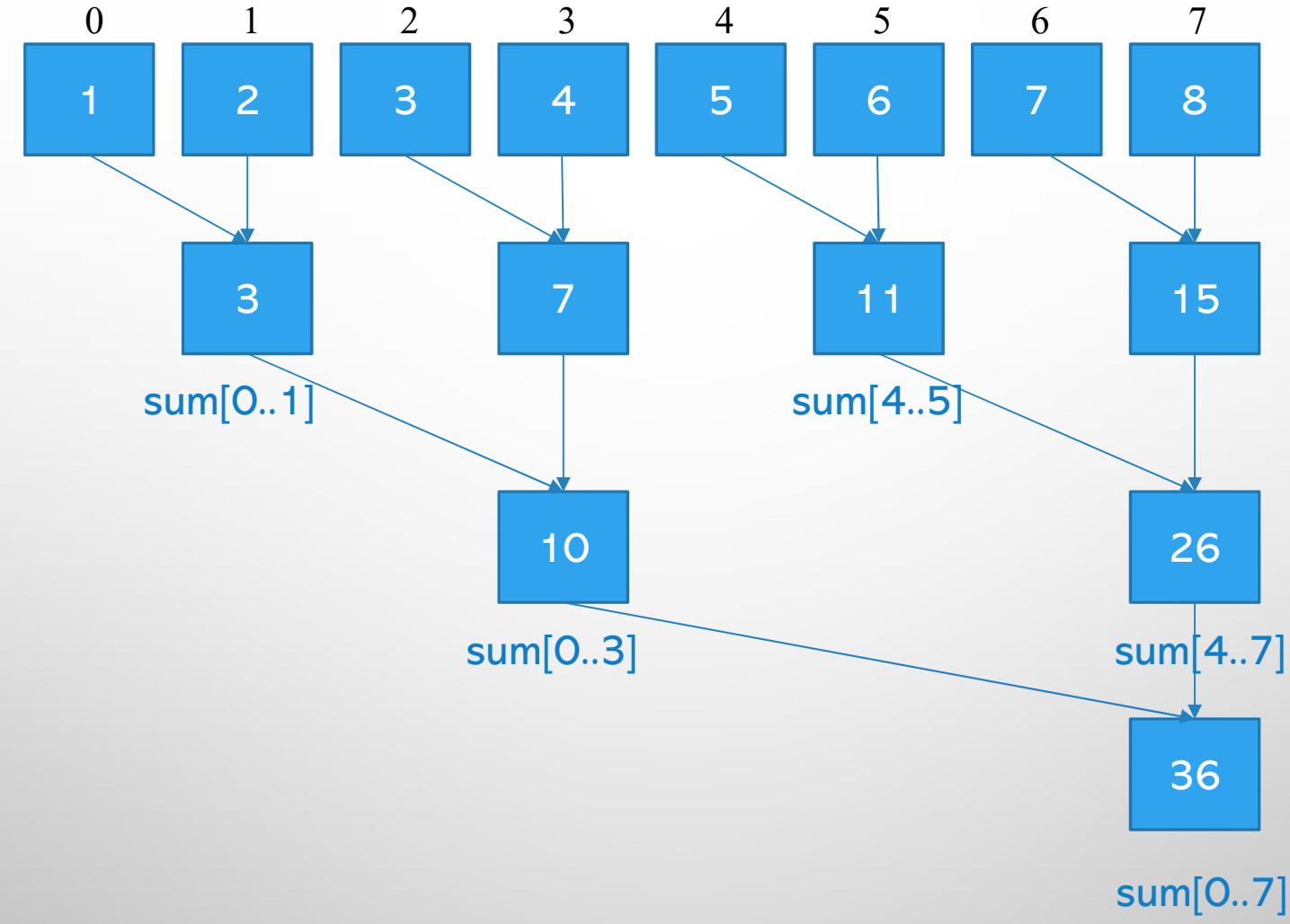
	Kogge-Stone	Blelloch
Time Complexity	LogN	2logN
Computational Complexity	O(NlogN)	O(N)
Speedup with P cores (P << N)	O(N/P logN)	O(N/P)

BRENT-KUNG PARALLEL SCAN



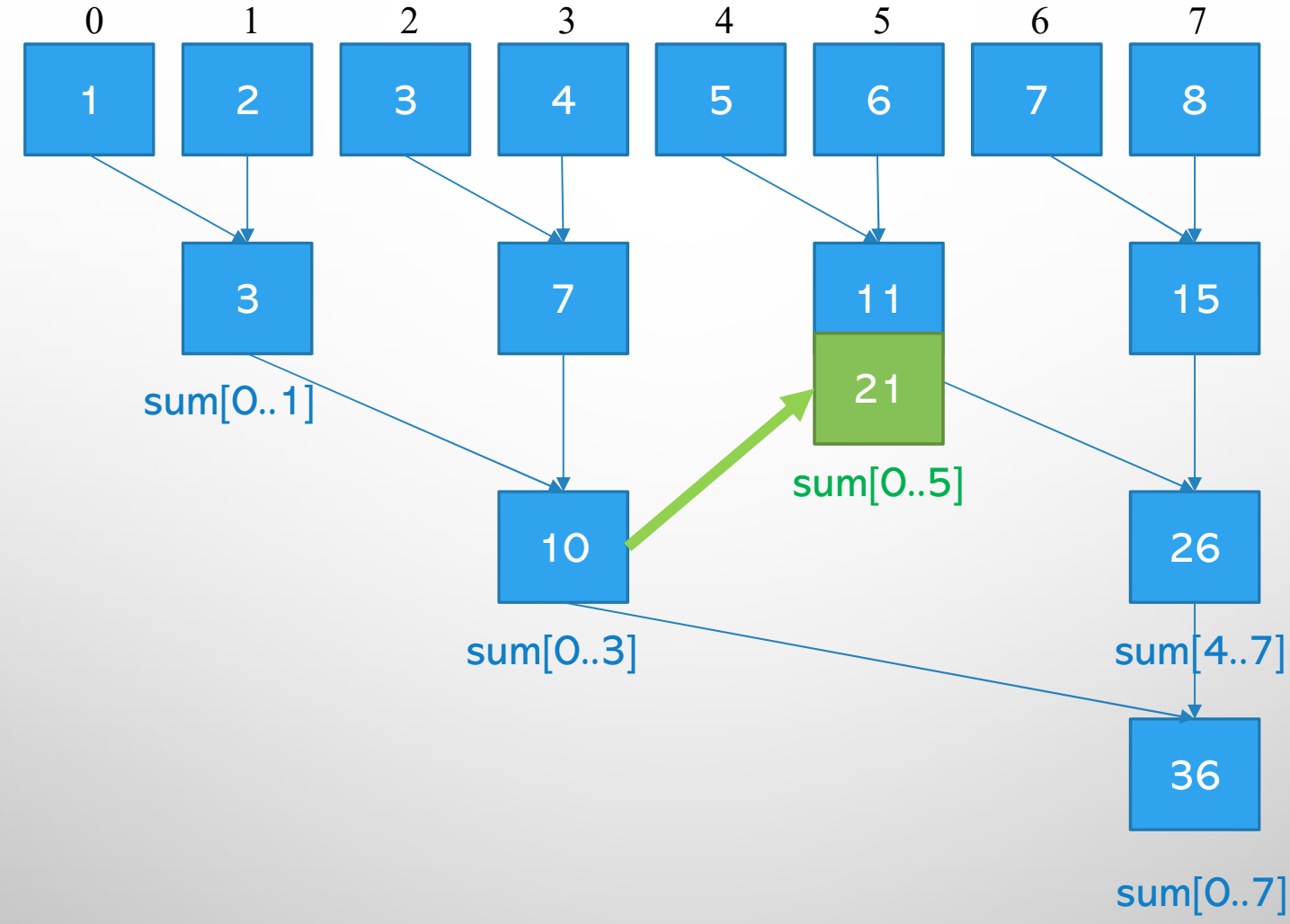
- Phase 1
  - Same as the parallel reduce (Just like Blelloch Scan)

BRENT-KUNG PARALLEL SCAN



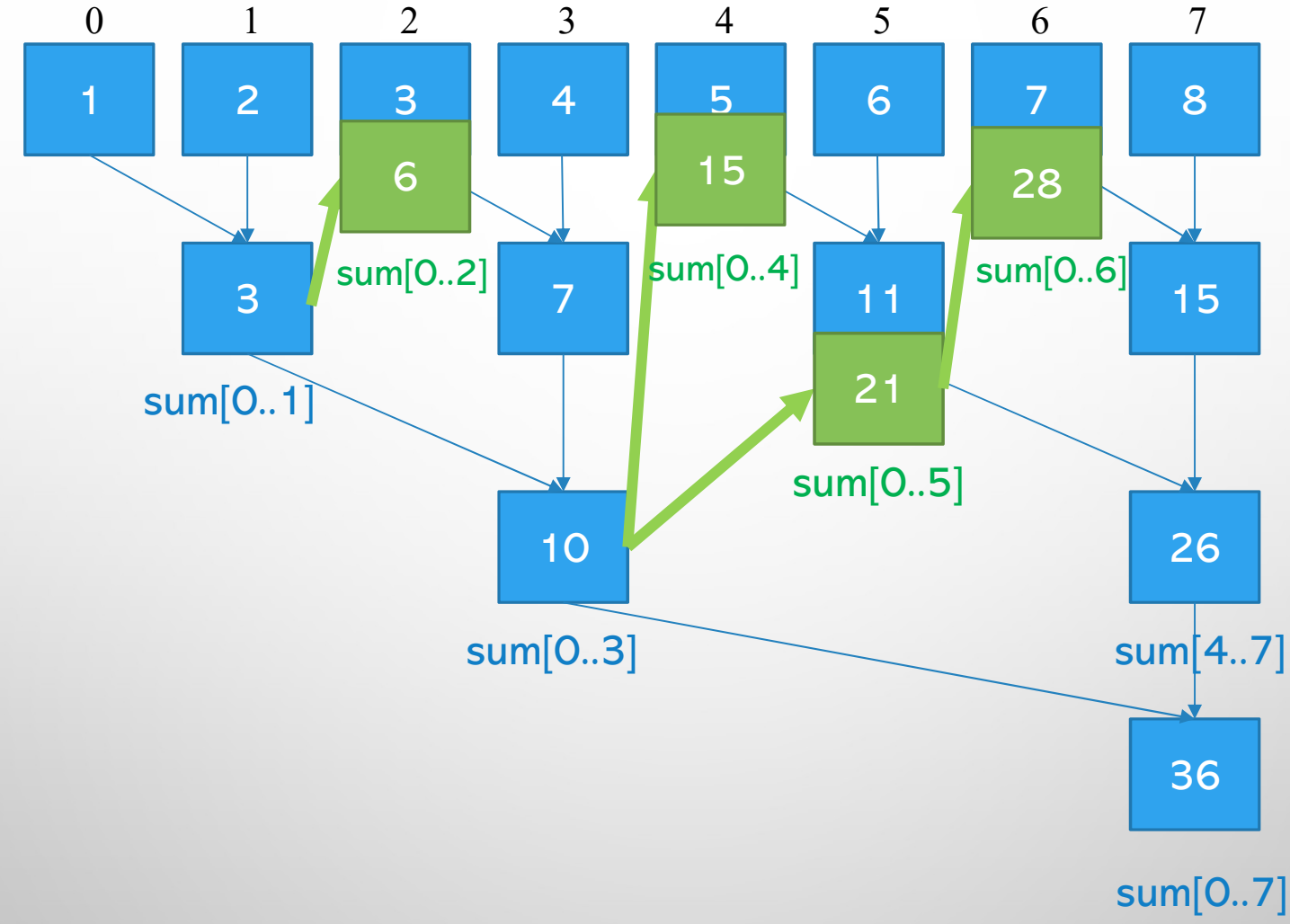
- Phase 1
  - Same as the parallel reduce (Just like Blelloch Scan)

BRENT-KUNG PARALLEL SCAN



- Phase 2
  - Topdown in a slightly different way

BRENT-KUNG PARALLEL SCAN



- Phase 2
  - Topdown in a slightly different way

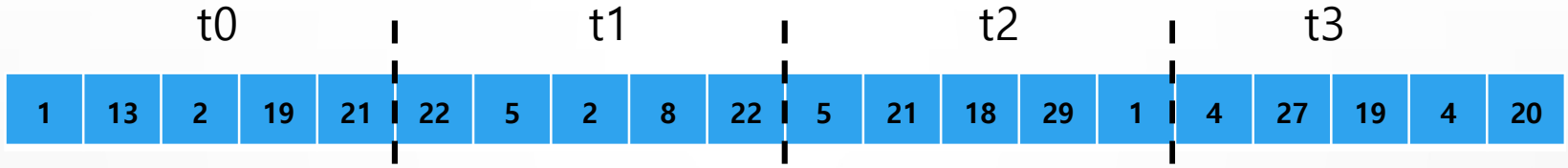
RANGE PARTITIONING – AN EXAMPLE OF PREFIX SUM USAGE

1	13	2	19	21	22	5	2	8	22	5	21	18	29	1	4	27	19	4	20

- We want to partition the numbers into bins – in this example, range of 10's

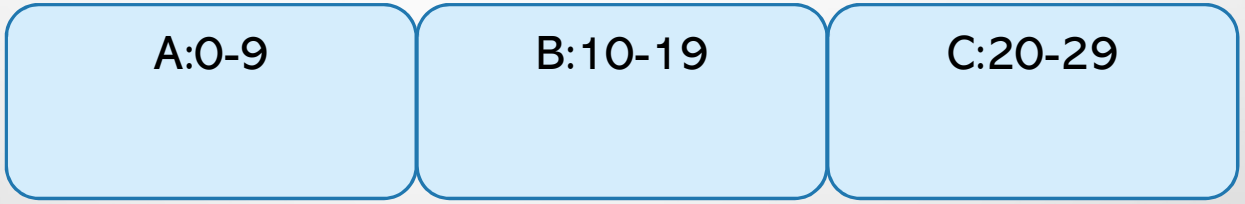
A:0-9	B:10-19	C:20-29
1 1 2 2 4 4 5 5 8	13 18 19 19	20 21 21 22 22 27 29

RANGE PARTITIONING – AN EXAMPLE OF PREFIX SUM USAGE

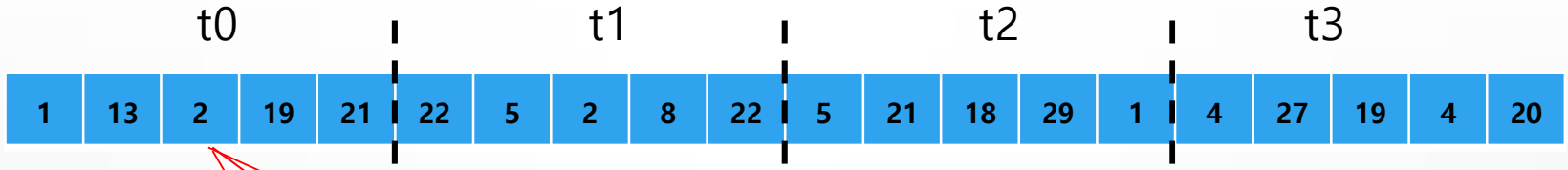


Distribute input array to threads (static allocation)

- We want to partition the numbers into bins – in this example, range of 10's

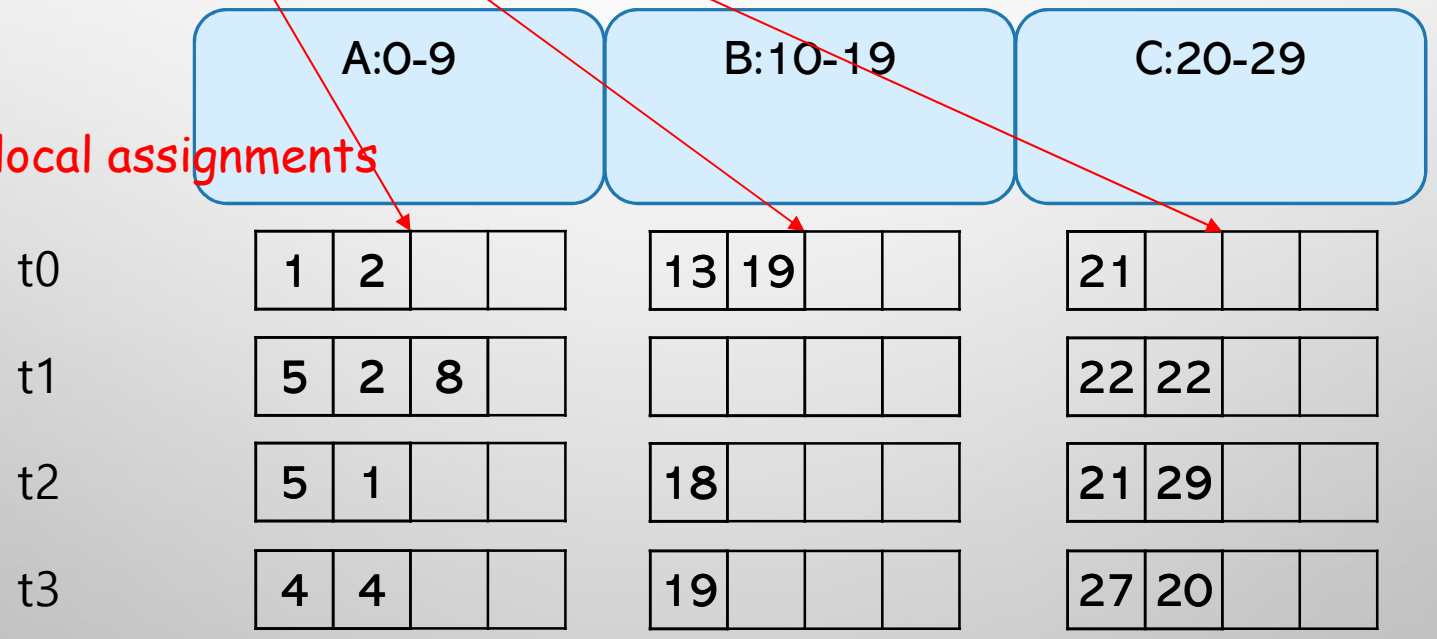


RANGE PARTITIONING – AN EXAMPLE OF PREFIX SUM USAGE

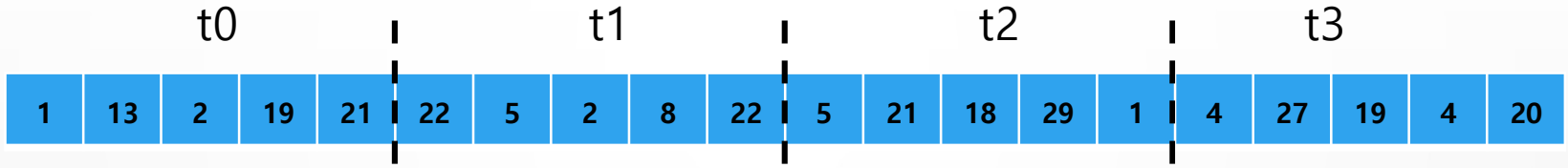


- We want to partition the numbers into bins – in this example, range of 10's

Perform local assignments

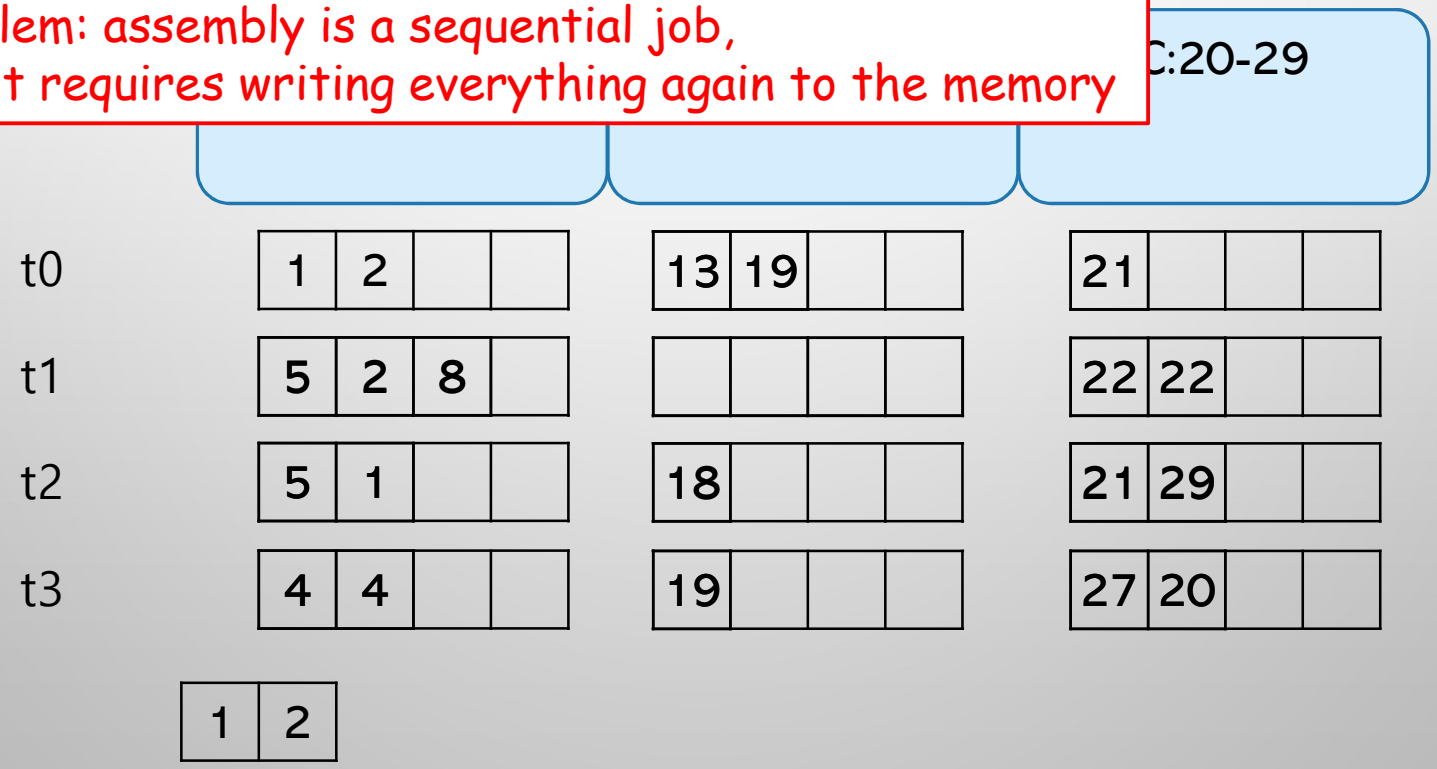


RANGE PARTITIONING – AN EXAMPLE OF PREFIX SUM USAGE

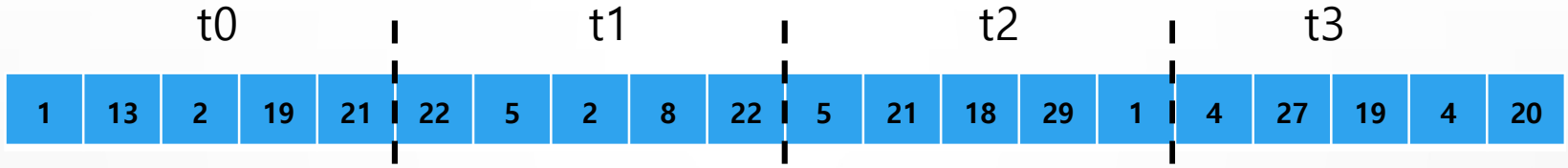


- We want to partition the numbers into bins – in this example, range of 10's

Problem: assembly is a sequential job,
and it requires writing everything again to the memory

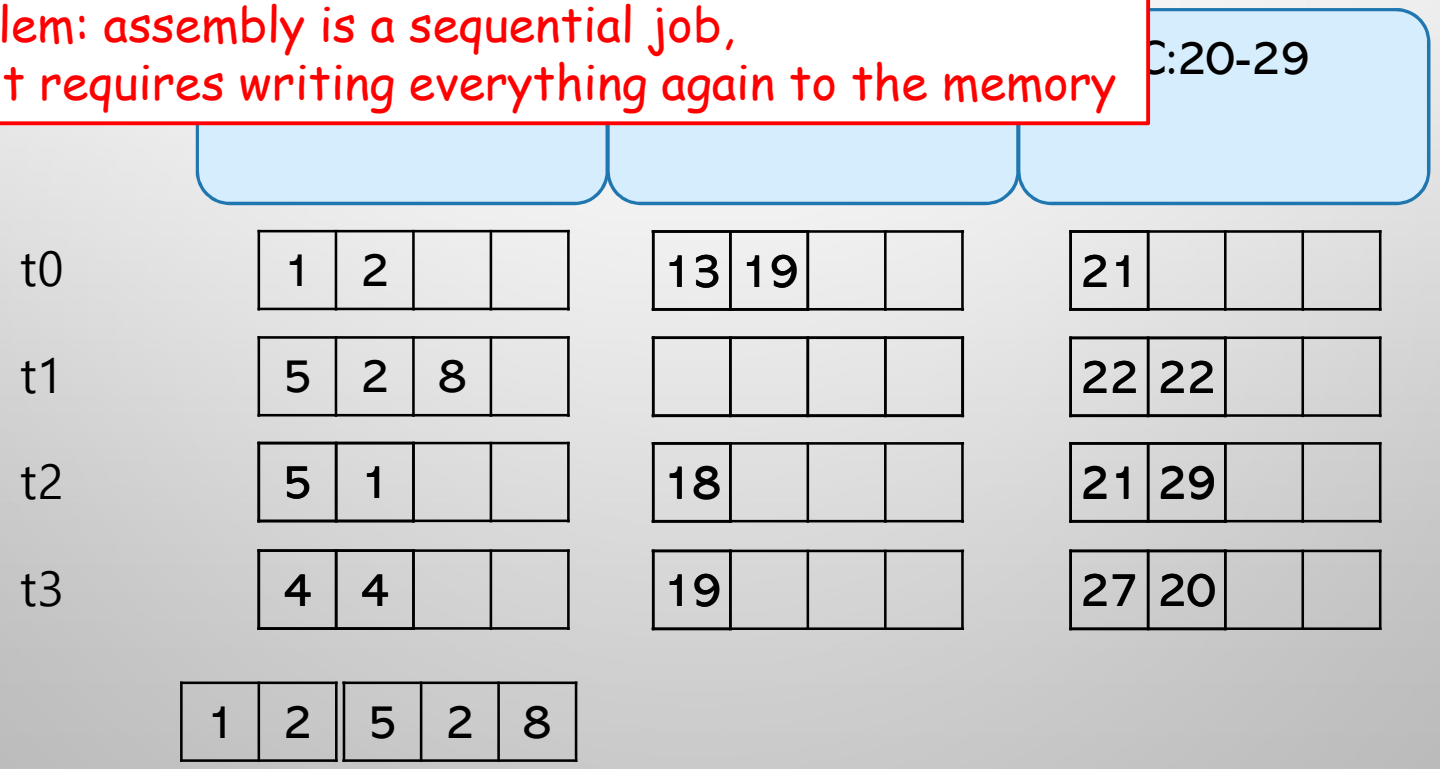


RANGE PARTITIONING – AN EXAMPLE OF PREFIX SUM USAGE

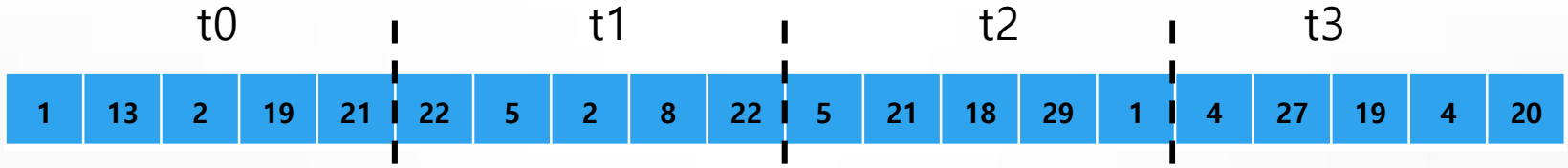


- We want to partition the numbers into bins – in this example, range of 10's

Problem: assembly is a sequential job,
and it requires writing everything again to the memory



RANGE PARTITIONING – AN EXAMPLE OF PREFIX SUM USAGE



- We want to partition the numbers into bins – in this example, range of 10's

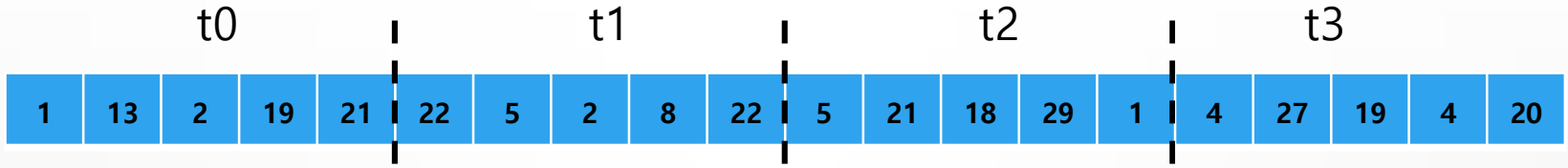
Problem: assembly is a sequential job,
and it requires writing everything again to the memory

C:20-29

t0	1 2	13 19	21
t1	5 2 8		22 22
t2	5 1	18	21 29
t3	4 4	19	27 20

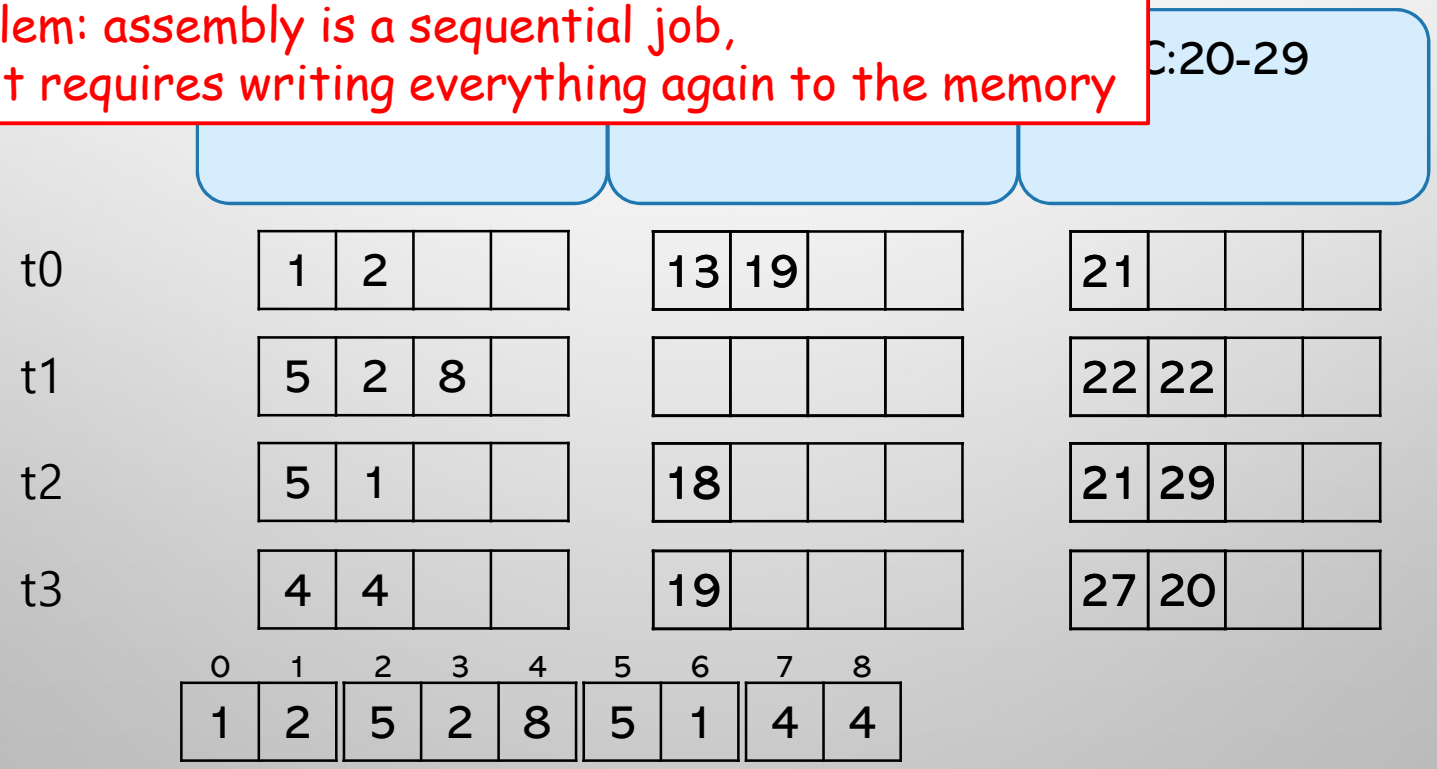
1 2 5 2 8 5 1

RANGE PARTITIONING – AN EXAMPLE OF PREFIX SUM USAGE

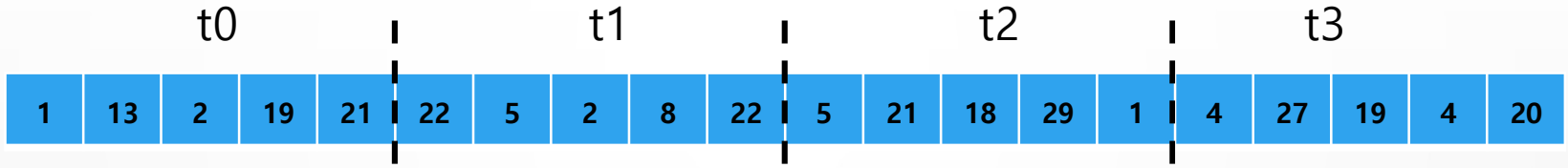


- We want to partition the numbers into bins – in this example, range of 10's

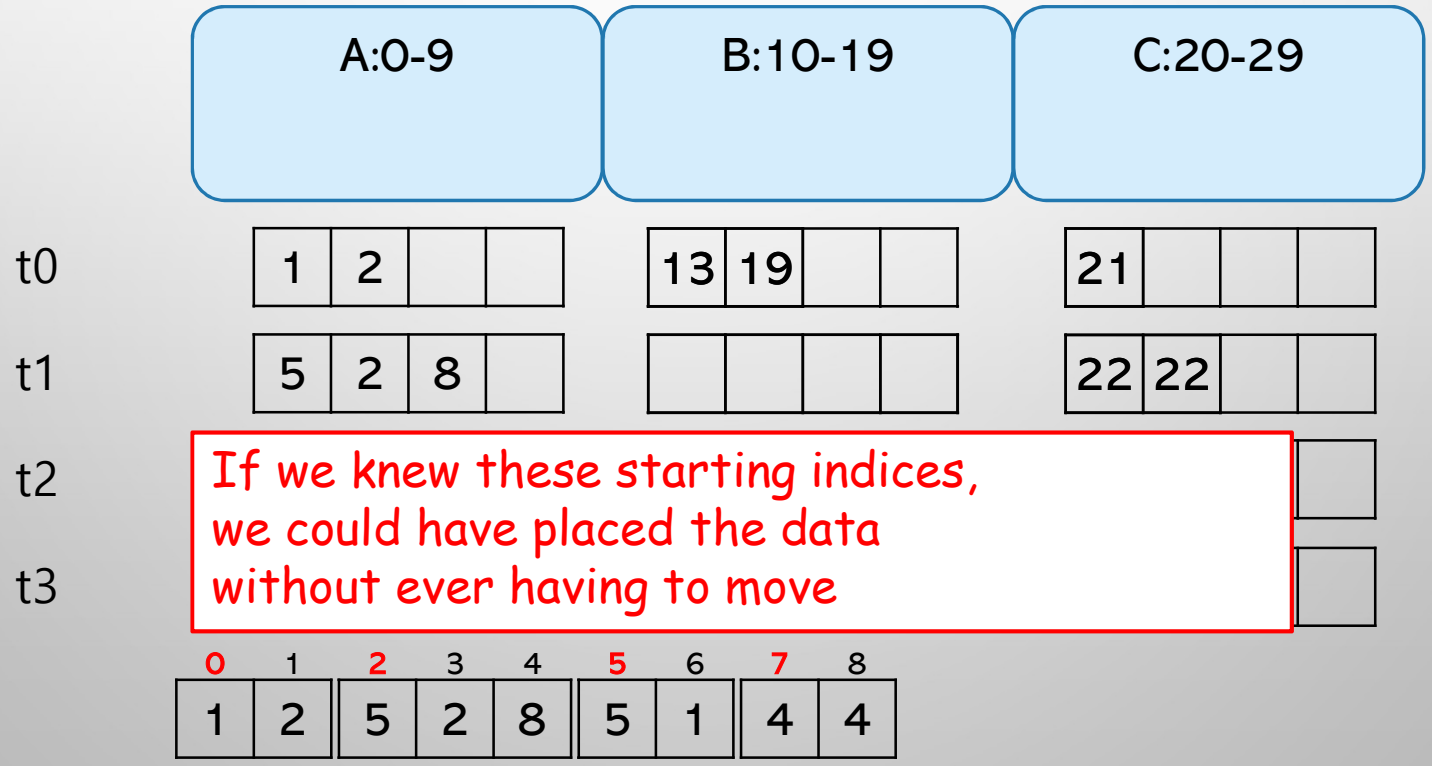
Problem: assembly is a sequential job,
and it requires writing everything again to the memory



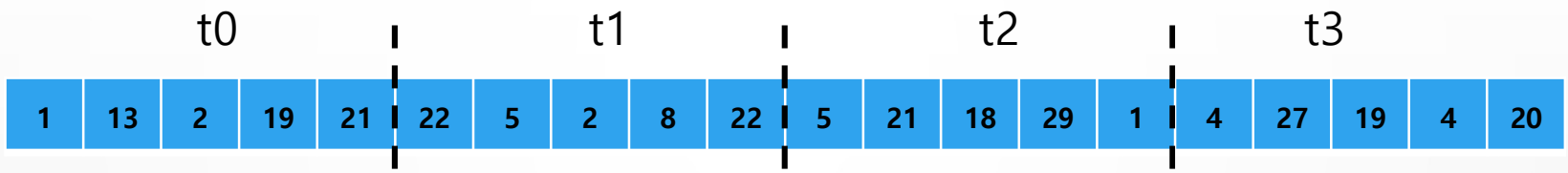
RANGE PARTITIONING – AN EXAMPLE OF PREFIX SUM USAGE



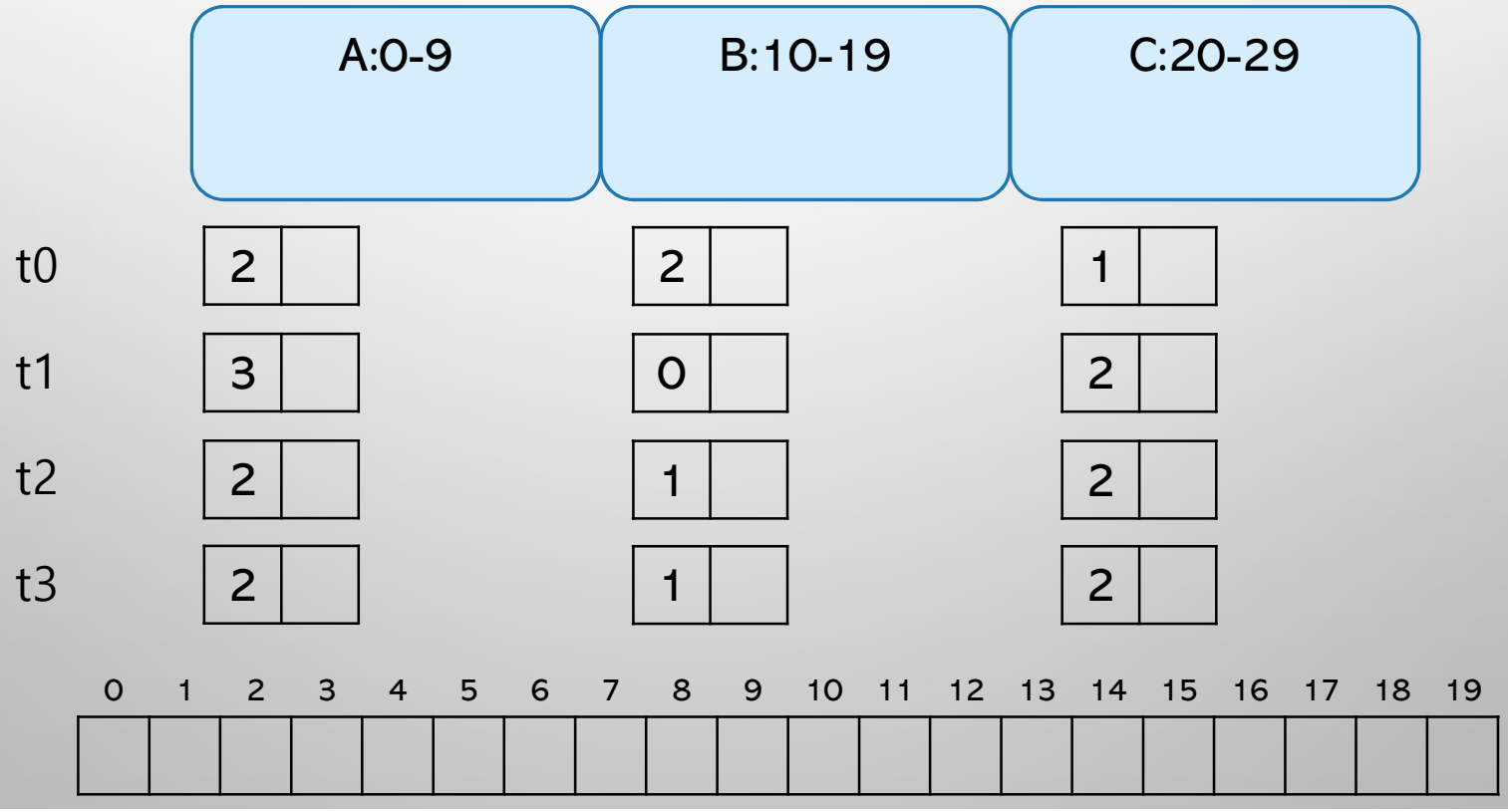
- We want to partition the numbers into bins – in this example, range of 10's



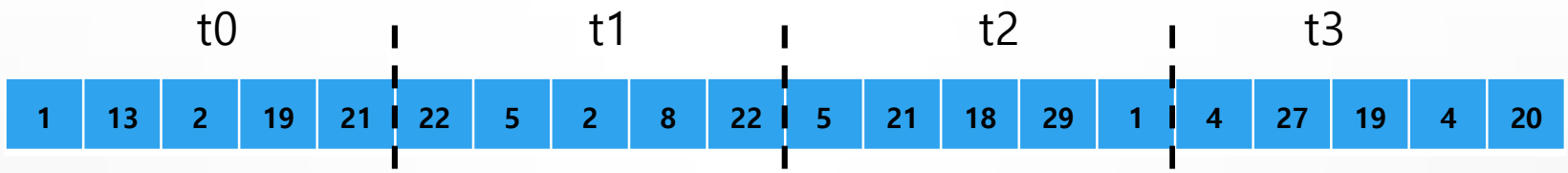
RANGE PARTITIONING – FULLY PARALLELIZED



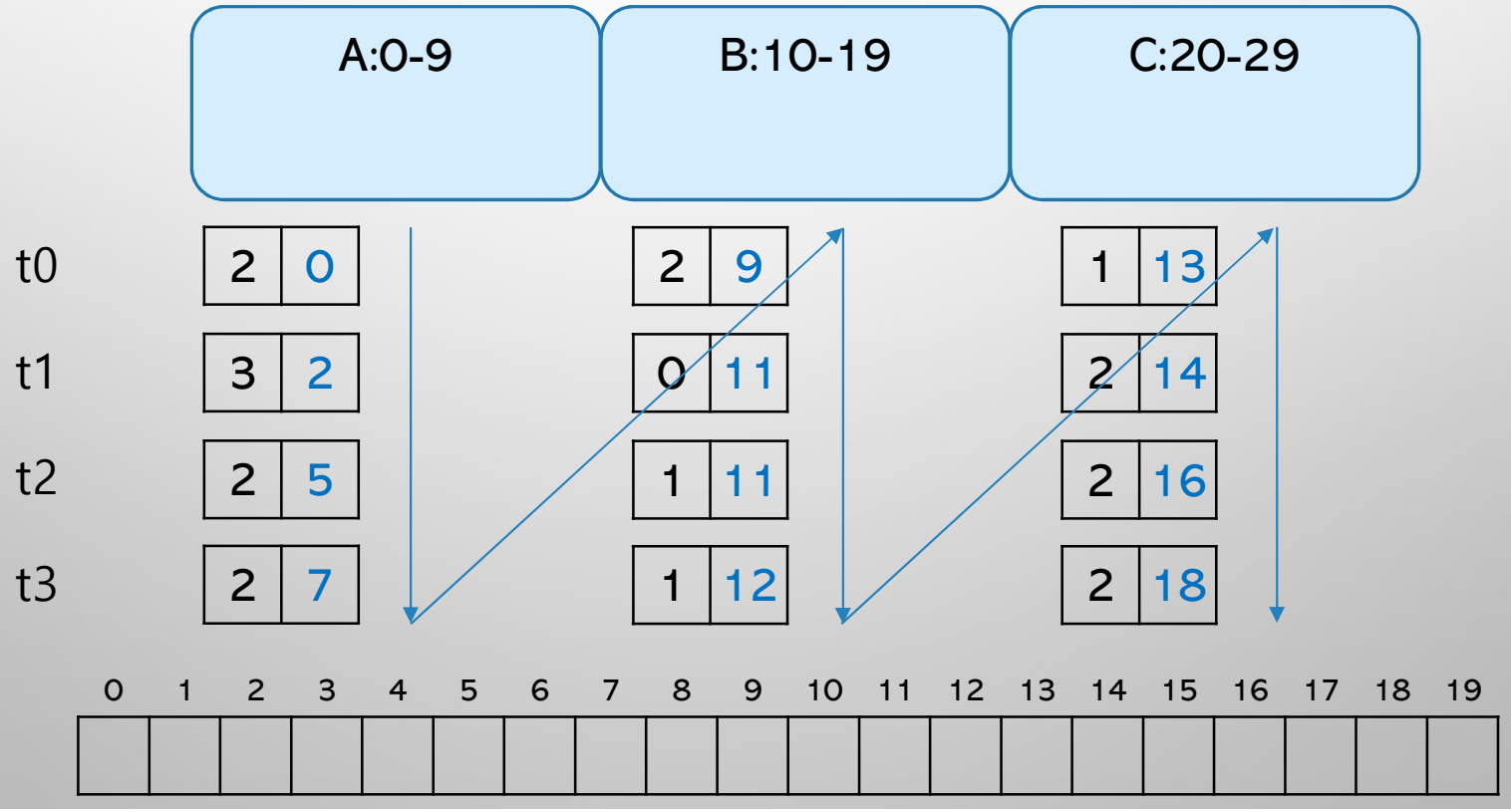
- Phase 1: Just count how many. Do not move items



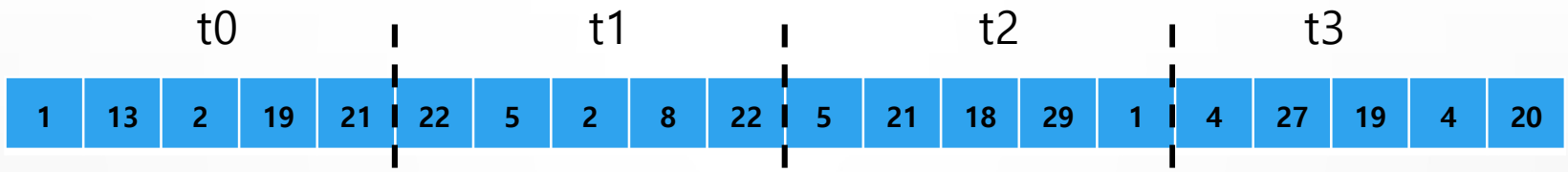
RANGE PARTITIONING – FULLY PARALLELIZED



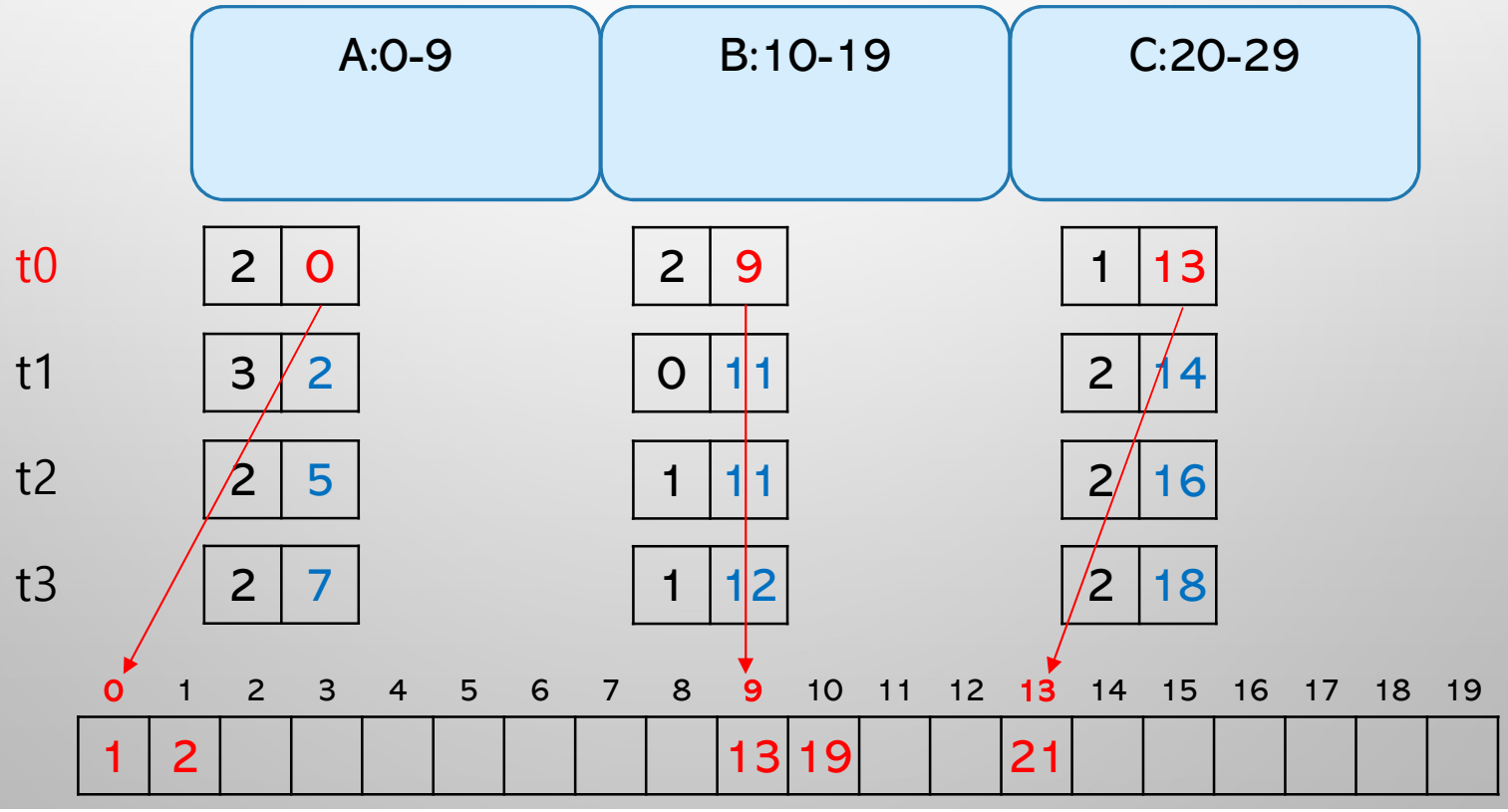
- Phase 1: Just count how many. Do not move items
- Phase 2: Prefix sum



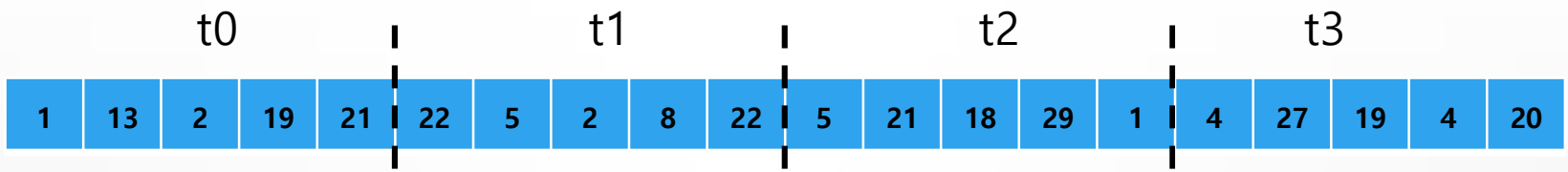
RANGE PARTITIONING – FULLY PARALLELIZED



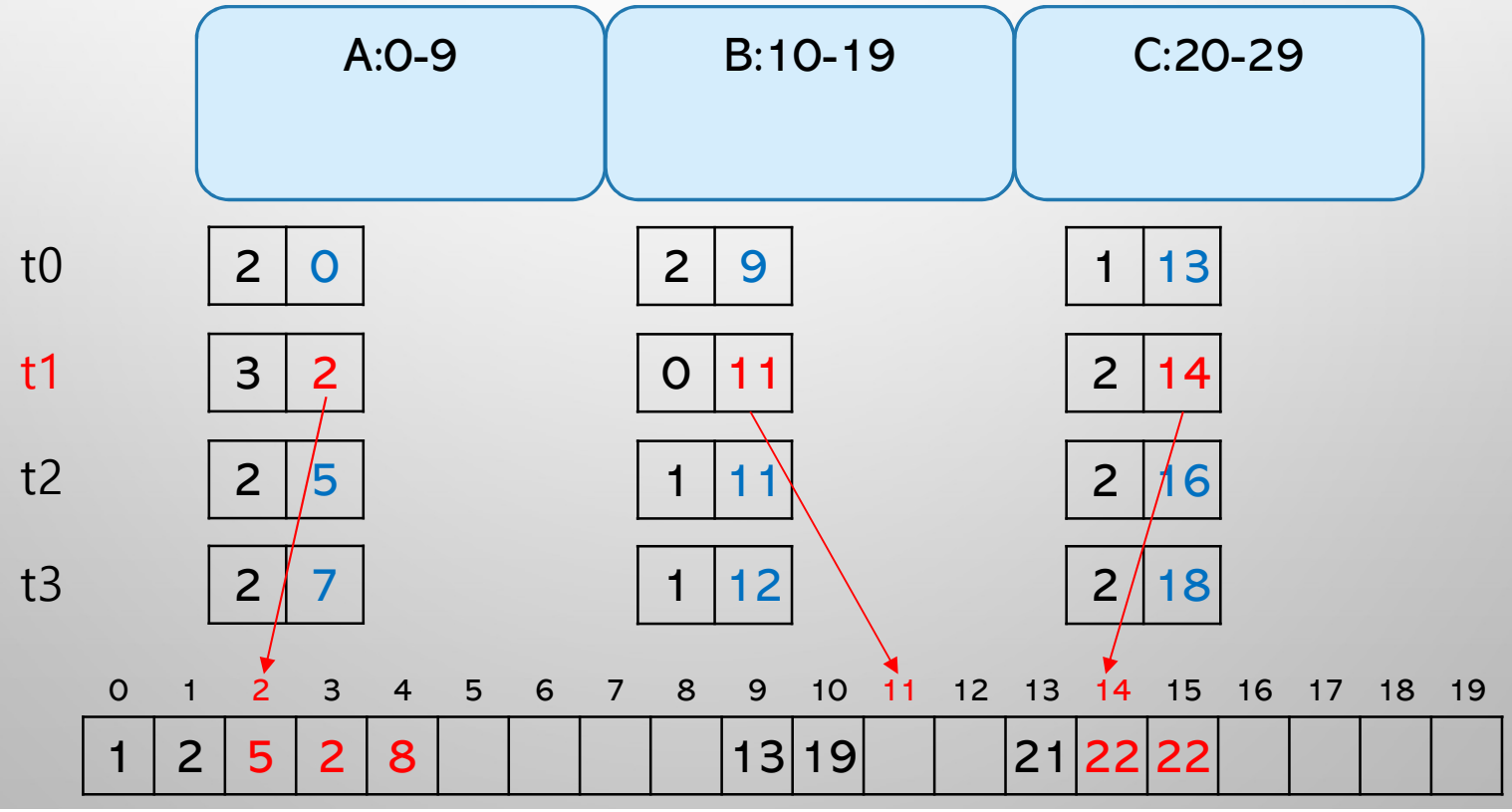
- Phase 1: Just count how many. Do not move items
- Phase 2: Prefix sum
- Phase 3: Perform partitioning in parallel without conflicts



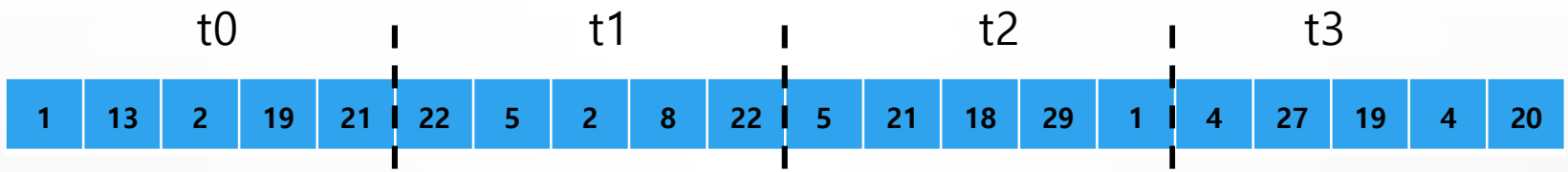
RANGE PARTITIONING – FULLY PARALLELIZED



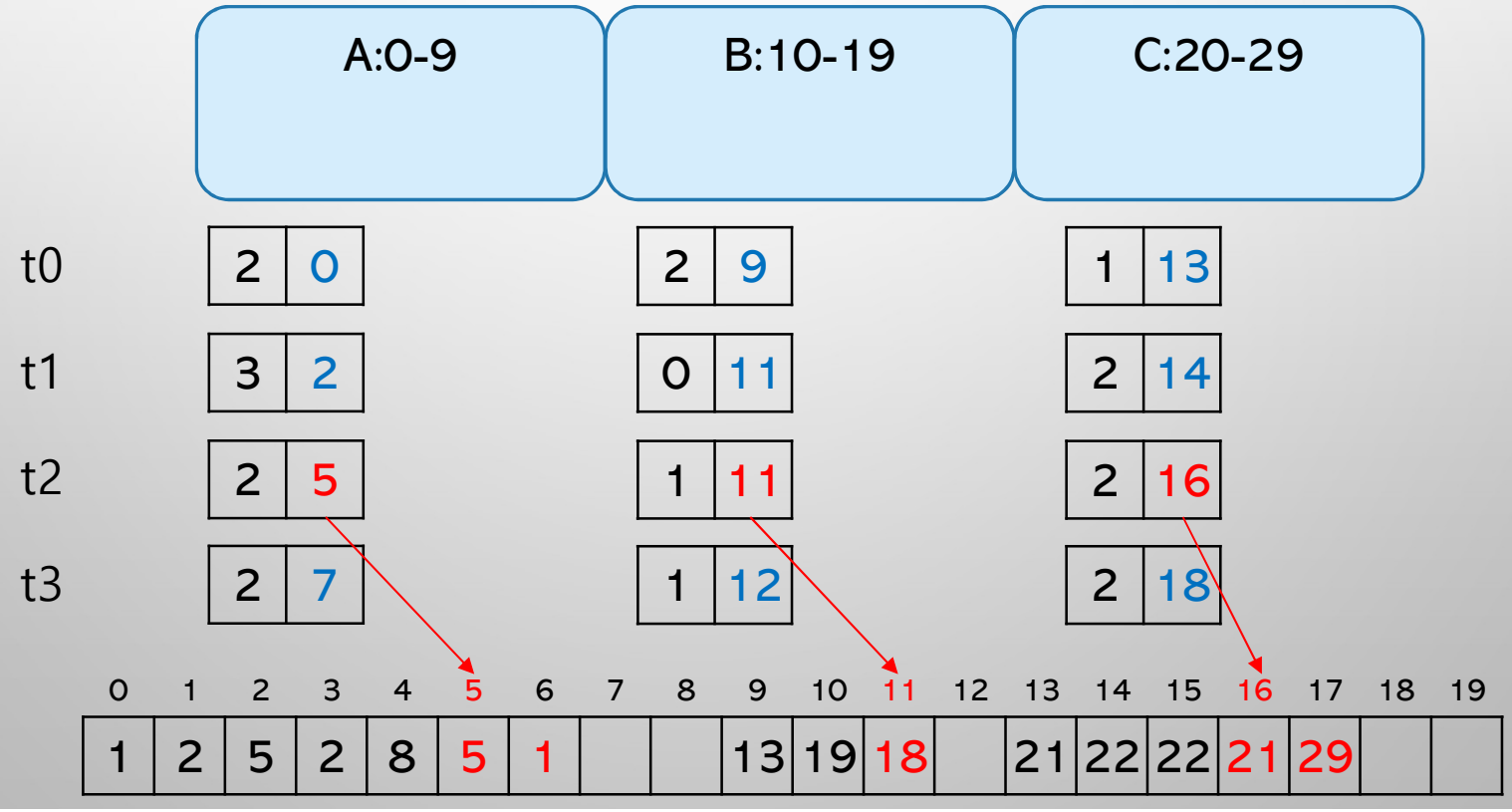
- Phase 1: Just count how many. Do not move items
- Phase 2: Prefix sum
- Phase 3: Perform partitioning in parallel without conflicts



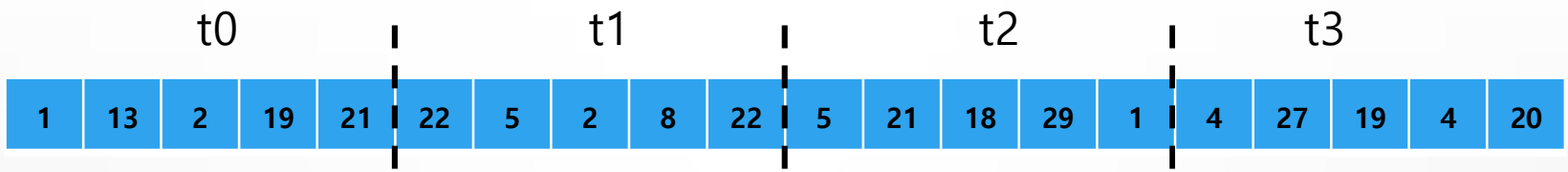
RANGE PARTITIONING – FULLY PARALLELIZED



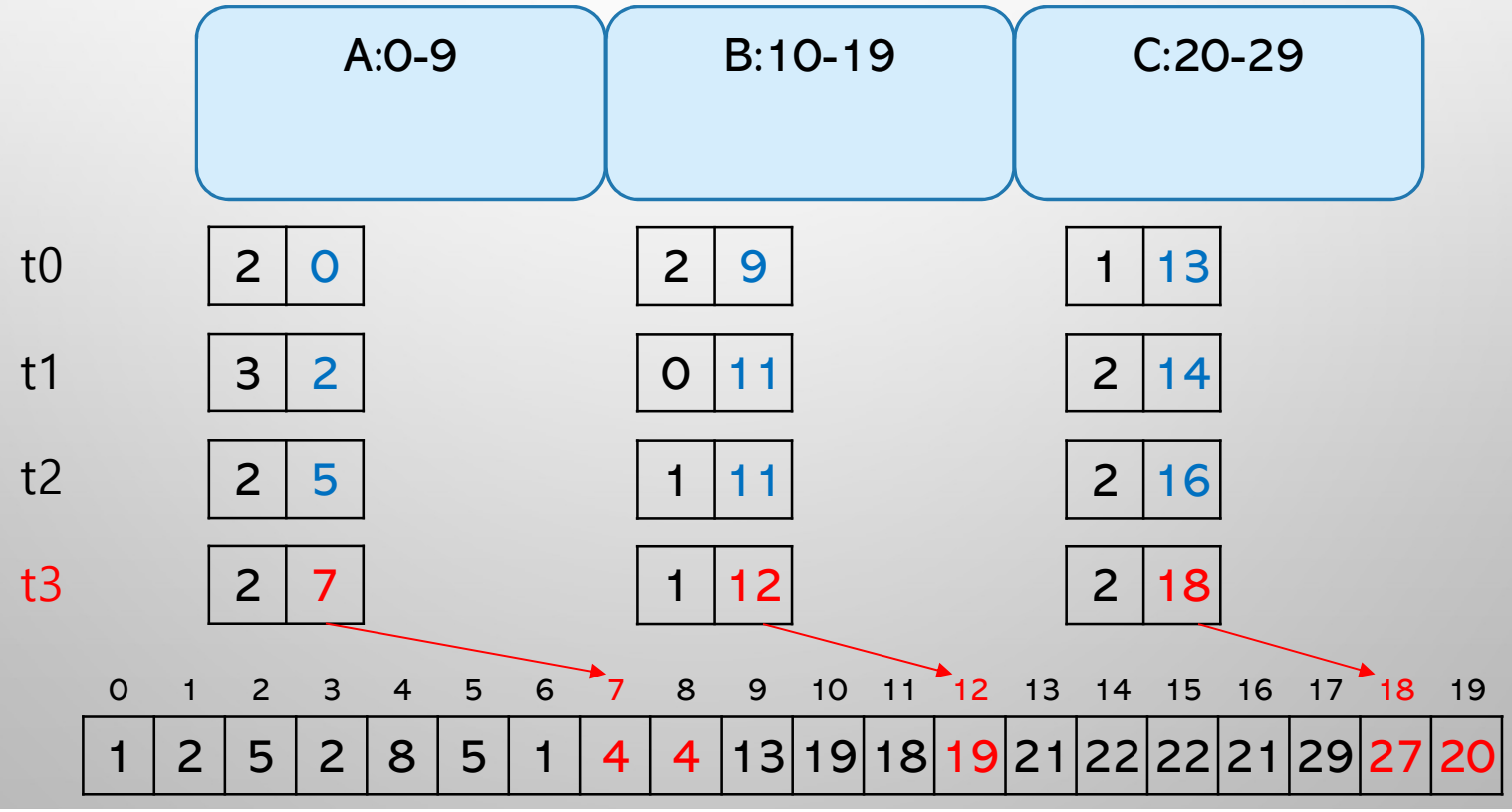
- Phase 1: Just count how many. Do not move items
- Phase 2: Prefix sum
- Phase 3: Perform partitioning in parallel without conflicts



RANGE PARTITIONING – FULLY PARALLELIZED



- Phase 1: Just count how many. Do not move items
- Phase 2: Prefix sum
- Phase 3: Perform partitioning in parallel without conflicts



SUMMARY

- Prefix sum requires some trick to parallelize
  - Kogge-Stone when you have enough hardware
  - Blelloch/Brent Kung when you don't have enough hardware
- It's difficult, but useful once you achieve parallelization
- Homework: How do you parallelize quicksort?